github.com/ssofiaavila

Stalling y Beltrán

Arquitectura de computadoras

Risc - Cisc
hay 2 tipos de arq

Clase 1

Concepto de programa

Antes: Datos → [Secuencia de fx orit. /lógicas] → Rtdos / Programax en hardware: al cambiar tareas, cambia hardware

Intérprete de inst + cód. inst + señales de control → implementation

Ahora: Datos → [Fx orit./lógicas] → Rtdos / Programax en software: en c/ paso, se efectúa alguna op sobre D.

- P/ c/ paso se necesita un nuevo conjunto de señales de control
- Las inst. proporcionan esas señales
- Nuevo concepto de programax

Arquitectura Von Neumann

- CPU formada por UC y ALU
- Datos e inst. deben introducirse en el sist. y rtdos se proporcionarán c/ componentes E/S
- Necesario almacenar temporal" D e instrucciones en: memoria ppal.
  - PC: contador de program
  - IR: reg. de instx
  - MAR: reg. dir. mem.
  - MBR: reg. buffer mem.
  - E/S AR: reg. dirx de E/S
  - E/S BR: reg buffer de E/S.

Repertorio de instrucciones

Conjunto completo de instrucciones que se realizan en una CPU.
Representado x un conj. de código de ensamblaje.
- De op: ADD, SUB, LOAD
- De operandos: ADD, Bx, PEPE

Elem. de una instrucción

- Cod. operax
- Ref a op fuentes
- " rtdo.
- " sig. instrux.

Almacenamiento de operandos

- Mem ppal o mem. virtual o mem. caché
- Reg. CPU
- Dispositivo E/S

Tipos de instx

- Procesamiento de D
  Inst. aritmético-lóg.
- Almac. de D
  Inst. de memoria
- Transf. de D
  Inst. de E/S
- Control
  Inst. de testeo y flujo.

Cant. direx

+ direx x inst: inst + complejos, + reg., - inst. en el program.

- " : inst. - complejos, + inst. en program., ejecuc de inst. + rápida

Decisiones en el diseño del conjunto de instrucciones

- Tipos de operandos
- Repertorio de op
  Cuántos, cuáles, complejidad
- Modos de direccionamiento
- Formato: longitud, nro direx, tamaño compos.
- Registros: nro. reg referenciables.

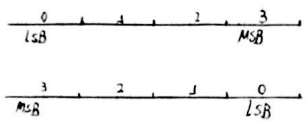
RISC contrapuesto a CISC → 2 tipos de arq.
- Computadora de conj. reducido de instx
- Computadora de conj. complejo de instx.

Scanned with CamScanner

github.com/ssofiaavila

Tipos de operandos
- Direcciones
- Nros
  - PF, pto flotante
- Caracteres
  - ASCII, EBCDIC, etc.
- Datos lógicos
  - Bits (reg flags)

Orden bits
Ej: 98765432H (32 bits) se puede almacenar en 4 byte de 2 formas: Big endian / Little endian



Big endian: el byte + significativo en lo direc c/ valor + bajo
Little endian: el byte + significativo en lo direc c/ valor + bajo

Dir byte	Forma 1	Forma 2
00	98	32
01	76	54
02	54	76
03	32	98

Tipos de operaciones
1. Transf. D
2. Aritméticas
3. Lógicas
3. Conversión
4. E/S
5. Control del sist.
- Control flujo.

1) Especifica: dirección op fuente y dest / función D / modd.
Dif mov → dif instr.

2) Op básicas (sum, RL, con o sin signo o PE) u otras op. como inc, dec, neg, obs/valor, shift left/right.

3) Op q' manipulan bits (booleanas, rotate left/right) / op p/ cambiar formatos (ASCII → EBCDIC)

4) IN/OUT - MOV - Se puede hacer mediante un controlador o puerto.

5) Modifican el valor contenido en PC
- Salto incondicional - JMP
- Salto condicional - Jz
- Salto c/ retorno o llamada a subrutina - CALL/RET

MODO
- Inmediato
- Directo de mem / absoluto.
- Directo de reg.
- Indirecto de memoria (en desuso)
- " c/ reg
- " c/ desplazamiento basado, indexado o relativo al PC
- Pila

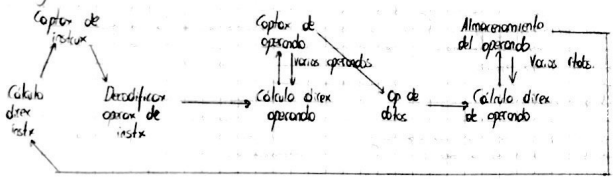
Ciclo de inst básico
Captar y ejecutar.

Ciclo de captar: la direc de inst. q' se debe captar se encuentra en PC. La UC capta la inst. desde memoria y la al IR. PC incrementa. UC interpreta inst y la llama a ejec

Ciclo ejecutar: acciones posibles
- Procesador - memoria
  - Transf. D. de CPU - Memoria
- Proc. - E/S
  - Transf. D CPU y módulo E/S
- Procesamiento D
  - op aritmética
- Control
  - Alterar secuencia - inst. de salto.

o alguna combinación de todas.

Diagrama de estados



Subrutinas
Inician en lenguaje de programación. Puede invocarse desde cualquier pto del programa. Brinda economía (cód. usado varias veces) y modularidad. Requiere pasaje de parámetros.

Pasaje de parámetros
- Vía registros: el nro. de reg es lo ppal limitador, se debe documentar reg. usados.
- Vía memoria: usa área definida de memoria, difícil de estandarizar.
- Vía pila: método + usado. Independiente a memoria y registros.

Funcionamiento de una pila
El operando se encuentra en la cabeza de la pila. Requiere un registro Puntero de Pila (SP / stack pointer) q' contiene la direc de la cabeza de la pila.
Operar sobre la pila: push apila
pop desapila. Son inversas e/ sí

Operaciones de apilar/desapilar
Secuencia de 2 acciones: mov. de dato (Reg-Mem o Mem-Reg.
modificar el' puntero antes/después de la anterior.

!! Tener en cta. donde apunta el puntero / cómo crece la pila

Definir del procedimiento
- nomb. proc
- ...
- ...
- RET
- nombre endp

(Cuerpo proc.)

Llamada al proc.
- ppal
- PUSH parametro1
- PUSH " 2
- CALL nombre

Posibles pasos en un procedimiento
1) Salvar BP (BP ingr)
2) SP
3) Reservar espacio p/ D locales (opcional)
4) Salvar val. de otros reg (")
5) Acceder a parámetros
6) Escribir sentencias o ejecutar
7) Retornar parámetros (")
8) Regresar al procedimiento

Ejemplo

```
        ORG 1000H
SUB     NEG AX
        RET
ORG 2000H  MOV BX,0
        MOV AX,dato
        PUSH AX
        CALL sub
        POP BX
        HLT
org 3000H dato  BB 55H
        END
```

Scanned with CamScanner

github.com/ssofiaavila

Pasos
1) Comienzo: c/ push BP
mov BP, SP

Establece a BP como puntero de referencia y se usa p/ acceder a parámetros y D de pila. SP no se puede usar. SP puede cambiar pero BP permanece quieto.

3) opcional

Se decrementa SP, reservando lugar. sub, SP, 2 (reserva 2 bytes p/ D locales)

5) 2 (tamaño de BP apilado) + tamaño de direc. de retorno + total parametros c/el buscado y BP.
p/ acceder a posición 1 = mov CX [BP + BX]

Salida del procedimiento
Los reg. salvados en la pila deben ser descargados inverso!. Si se reservo espacio p/ variables locales, se debe repetir SP c/el valor de BP q/ no cambio en proc.
Restore BP.
Volver al programa c/ RET

2) La 1ª inst. salva BP y la 1º carga a SP en BP.
BP es el puntero al area de la pila asignado al procedimiento.
p/ acceder a los D se deberá sumar un desplazamiento fijo a BP

4) opcional
Ej: push DI
si el proc. no cambia val. de reg. no necesitan ser salvados.

```
POP DI
mov SP, BP
POP BP
ret.
```

En 8086 se usa BX en vez de BP

Bibliografía W. Stallings: cap 9/10 / apéndice 9A / cap. 3 / 11 / 10.

Practica 1

1)

Instrucción	Valor SP	AX	BX
mov Ax, 5	Pcc H	5	
mov bx, 3	Pcc-4	5	3
PUSH Ax	7FFE	5	3
PUSH Ax	7FFC	5	3
push Bx	7FFA	5	3
POP Bx	7FFC	5	3
POP Bx	7FFE	5	5
POP Ax	Pcc H	5	5

2)

Instrucción	Valor SP
org 200h rutina: mov bx, 3 ret org 200h push ax call rutina pop bx hlt end	7FFC 7FFE 7FFE FCC

3) a)
```
org 200h
mov bx, 3
ret
org 200h
call rut
add cx, 5
call rut
hlt
end
```

b) 1) call rut lo pop ...

ii) ref. retorno ...

4)

Código	Registro	Pila	Valor	Referencia
mov bx, 5 call subrutina				
mov dx, offset A call subrutina	✓			✓
mov bx, 5 push bx call subrutina pop bx		✓	✓	
mov cx, offset A push cx call subrutina pop cx				
mov dl, 5 call subrutina				
call subrutina mov A, dx				

5) Cálculo A+B=C.

a)

b)
```
org 200h
...
mov dx, offset A
mov cx, 0
...
HLT
END
```

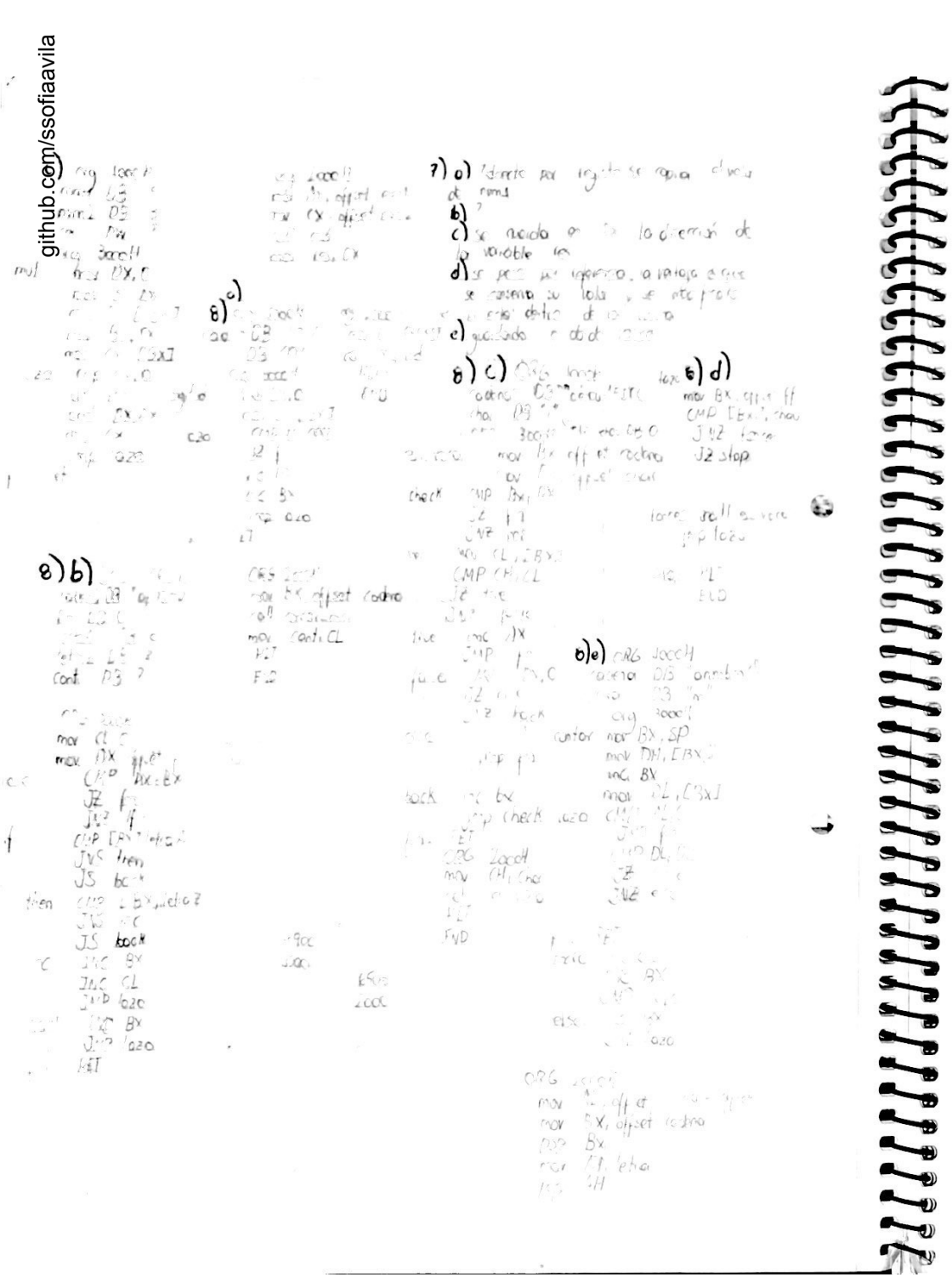
c)
```
org 200h
...
mov bx, B
mov cx, C
...
mov D, bx
```

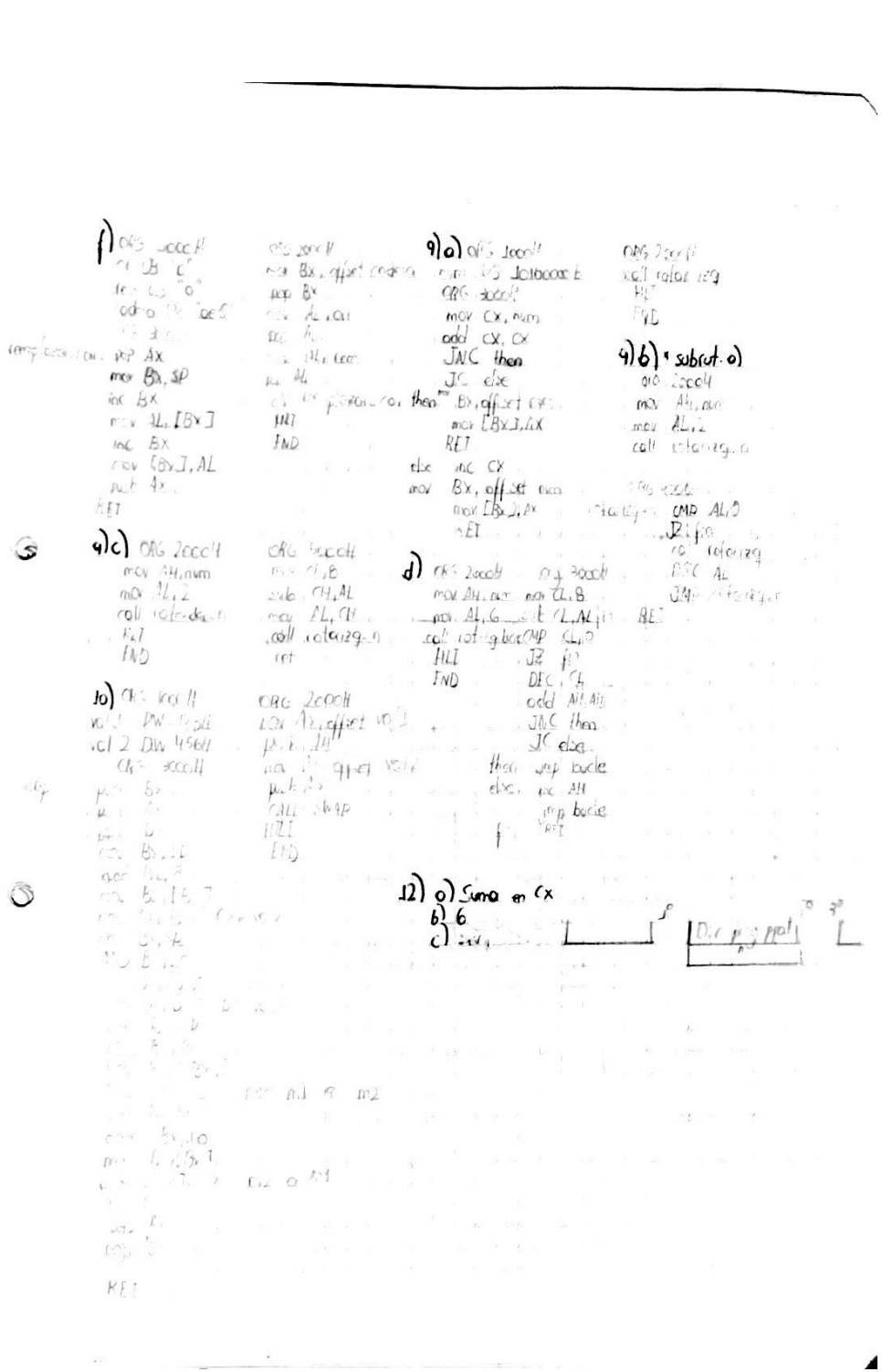
d)

6) a)
```
org ...
num ...
res DW ?
org 200h
...
mov B, ...
mov ...
...
```

Scanned with CamScanner

github.com/ssofiaavila





Scanned with CamScanner

github.com/ssofiaavila

11) a)

b)

c)

Clase 2 • Mecanismo mediante el cual se puede interrumpir el procesamiento normal del CPU. q' tiene una ejecux secuencial de instrucciones.

La interrupción puede ser de origen interno o externo al CPU

Motivos de interrupx: Fallo de una ejecux de una instrux. (Ej: V o, div 0)
- temporizador interno del procesador (Permite al SO hacer fx regular)
- Qq de E/S (p/ indicar la finalización de una op.)
- Fallo de hardware (Error paridad de memoria, pérdida energía).

Qué hacer si interrumpen?
En casi todos los casos implica transferir el control a otro programa (Gestor) que:
- Salve E procesador
- Corrija / responda motivo q' ocasionó la interrupx
- Restaure E original procesador.
- Retome ejecux normal program interrumpido.

Jerarquía de interrupciones
Si hay multiples que pueden solicitar interrupx se establece cuáles son + importantes. Se consideran:
- No enmascarables: NO pueden ignorarse. Son eventos peligrosos o de 1ª prioridad.
- Enmascarables: pueden ser ignorados c/ instrux. podemos inhibir la solicitud

Interrupciones por hardware
Generados por dispositivos de E/S. Son "verdaderas" interrupciones. El sist. de cómputo tiene q' manejar eventos ext. "no planeados" o "asincrónicos". No relacionados c/ el progr. en ejecux en ese momento.
Conocido como interrupt request.

Flag I: es el que hace las interrupciones

Vectores identificados

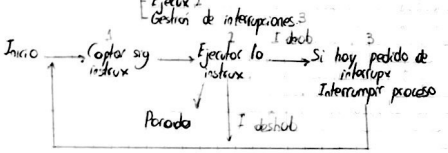
Tipos interrupciones
Interrupciones < hardware creados < el procesador en ltro. o ciertos eventos.
- Cond. excepcionales: V qq ALU de pto flotante.
- Fallo del program: tratar de ejecutar instrux no definido.
- " de hardware: error paridad de memoria.
- Accesos no tratados o a zonas de memoria protegidas.

Interrup < software:
Muchas progr. tienen instrux explícitas q' afectan al procesador de la misma manera q' las interrup. < hardware.
Gral' usados p/ hacer llamados a fx del SO.
No requieren conocer direx de la rutina en tiempo de ejecux.

Hay sist q' no permiten hacer una llamada directa a una direx. de la fx del SO x estar en zonas reservadas.

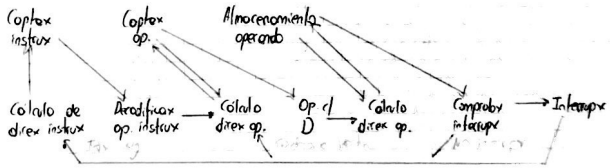
Si no tuviera interrupciones por software, debería escribir todas las fx q' necesito o al cargar un programa habría q' mirar fx de otros y SO y reemplazar el cod. de dx de fx.

Ciclo instrux 3) Captar 1
Ejecux 2
Gestión de interrupciones 3



Ciclo de interrupx
Se comprueba si se ha solicitado alguna interrupx. (Indicado x flag) → interrupt
Si no hay señal, se capta la sig instrux
Si hay algún pedido de interrupx pendiente:
- Se suspende ejecux en curso
- Guarda contexto (prox. instrux a ejecutar y E procesador).
- Carga el PC c/ direx comienzo rutina de gestión
- Finalizada la rut. de gestión el prox. retoma la ejecux del program.

Diagrama de E de ciclo de interrupciones



Scanned with CamScanner

github.com/ssofiaavila

Interrupx multiples

Interrupciones inhabilitadas
El procesador puede y debe ignorar la señal de petx de interrupx si se produce una interrupx en ese momento
Si se hubiera generado una interpx, se mantiene pendiente y se examinará luego una vez q' se haya habilitado nuevo.
Ocurre una interrupx, se inhabilitan, se gestiona la misma y luego se habilitan otra vez.
X lo tanto, las interrupciones se manejan en un orden secuencial estricto.

Definir prioridades
Una interrupx de prioridad + 1 puede interrumpir a un gestor de interrupx de prioridad x
Cuando se ha gestionado la interrupx de prioridad + 1, el procesador vuelve a las interrupx previas.
Terminados las rutinas de gestión, se retorna al program. del usuario.

Mecanismo de interrupciones

Interrupciones multinivel
c/ dispositivo q' puede provocar interrupx tiene una entrada física de interrupx conectadas al CPU. Sencillo pero raro.

Línea de interrupx única
Una sola entrada física de pedido de interrupx a la q' están conectados los dispositivos. Se debe "preguntar" a c/ disp. si ha producido el pedido de interrupx "polling/encuesta".

Interrupciones vectorizadas.
El disp. q' quiere interrumpir además de la señal de pedido de interrupx, debe colocar en el bus de D un identificador.
Lo coloca el periférico directo o controlador de interrupt.

Escenario de trabajo
Si el procesador tiene una única E de pedido de interrupciones.
Si tenemos varios productores de interrupciones se solucionan c/ el PIC "dispositivo controlador programable de interrupciones"

Interrup del MS-DOS
- Hardware: línea INT (cdo de reconocimiento INTA). / línea NMI (proc. de obtenx x solo ind.).
- Software: instx INT xx. // el retorno desde el gestor debe usarse la inst. IRET
- Proceso de obtenx vectorizado.

Tabla de vc. de interrupx
Es el enlace nexo c/ tipo de interx (0..255) y el proc. designado p/ atenderlo.
c/ E. es una doble palabra (4 bytes)
↳ Direx del próx. q' brinda el serv.

Vc. preasignados: 0= finaliza ejecux
3= pto de parada
6= lectura entrada. Requiere BX
7= escritura de S. " y AL

Software = rutinas que usan PIC ; hardware =

PIC - programable → mux y d pedidos

Configuración de interrupciones ?
Registros internos PIC *

Cop 3 Stallings

Coprocesador y direccionamiento
Los reg. int. del PIC se sitúan desde 20H. Se acceden c/ op de lect. y esc.
Interrupciones de hardware asignadas:

INT0 - FIJO
INT1 - timer
INT2 - handshake
INT3 DMA
INT4 o INT7 - no usados

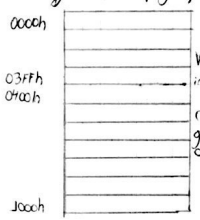
* Los reg int. del PIC se sitúan a partir de 20h.
Accedidos c/ op E/S. Int. hardware asignados:

EOI: p/ comandos escribir. p/ fin int. escribir 20H

- IMR: máscara de interrupx: enmascara c/1
- IRR: petx de int. Indica c/bit en 1
- ISR: int en serv.
- INT0 INT7: c/u c/su vc.

Práctica 2

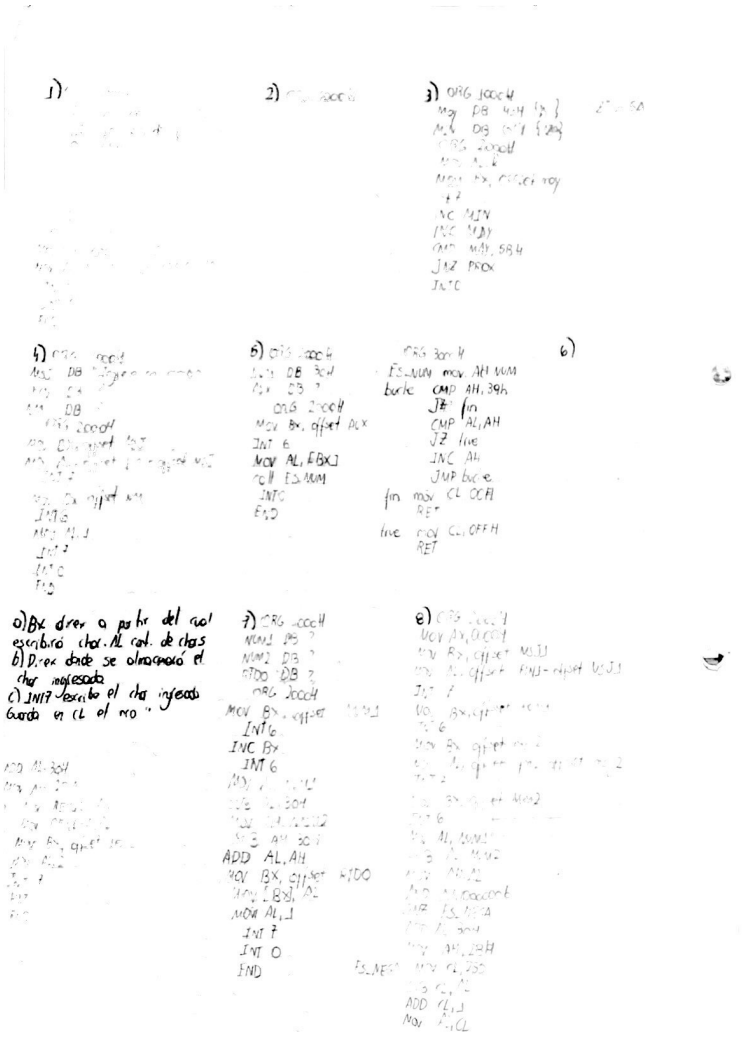
Cuando un disp quiere interrumpir debe enviar la sig info: señal pedido interrupx, y un identificador. Se usa el PIC, configurar interrup habilitadas y donde se encuentra su rutina.
Una interrupx ocurre en cualquier momento, antes de ir a la rutina de la interrupx, se guarda en pila: flags y IP (direx del retorno)

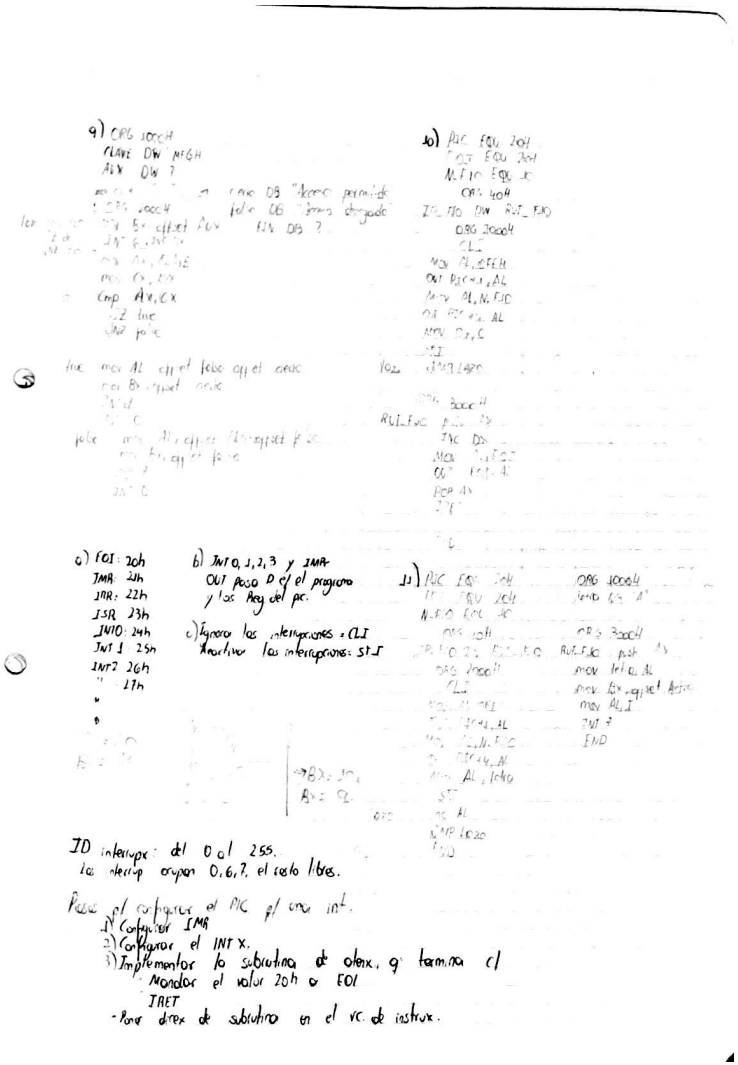


Interrupx software: usan posix del vc preasignados. NO pueden usarse p/ definir otras rutinas.
- INT0 → HLT
- INT 3 → Debug (pto de parada)
- INT 6 → leer char. en BX almacena direx
- INT 7 → escribe cadena de char.
  BX → direx inicio cadena.
  AL → cant. char a escribir.

Scanned with CamScanner

github.com/ssofiaavila





Scanned with CamScanner

github.com/ssofiaavila

12) Registros de reloj seguidores q marcan seg transcurridos desde apagar

14) Reloj q arranque y detenga su marcha al presionar sucesivas veces F10 y q finalice conteo a los 30"

Clase 3 Problemas de E/S. Existe una gran variedad de periféricos con varios métodos de operar.
Como transmiten dif cant de D en <> veloc. y usan <> longitudes y formatos de D
Son + lentos q la CPU y RAM. Hay una necesidad de módulos de E/S.

Un módulo de E/S realiza una interfaz el el procesador y la memoria (bus) y los periféricos.
Pueden manejar 1 o más periféricos

Dispositivos externos:
- E/S básicos: monitor, mouse, teclado
- Almacenamiento: disco duro, CD, DVD
- Impresión: impresora y scanner
- Comunicación: el disp remotos: modem, acceso interfaz de red
- Multimedia: micrófono, parlantes
- Automatización y control: sensores, alarmas, adquisición de D

Estado
Maneja el modulo de E/S

Control dentro modulo E/S

D desde y hacia el módulo E/S

Lógica de control

Buffer

Transductor

Datos
Desde y hacia el exterior

1) P/saber qué hacer con las señales
2) Manejo de D
3) Transductor: p/convertir D. Convierte señales eléctricas en algún tipo de D.

Hay una conexión mediante puertos

Funciones de un módulo de E/S.
- Controla y temporiza uno o + disp ext.
- Interpreta los órdenes q recibe de CPU y transmite al periférico.
- Comunica el CPU y memoria.
- Controla transf de D el CPU y periférico.
- Comunica el disp
- Informa o CPU E del periférico.
- Almacenamiento temporal de D
- Detecta errores.

Conect. de un puerto:
Interfaz el el periférico y el módulo E/S
c/ señales de control, E y D.
5 señal de control: fx a realizar.
4 ° de E: Ready/not ready
1 Control lógico: manejo de dispositivo.
3 Transductor: convierte de D.
2 Buffer: adaptar (1, 8, 16 bits)

Capacidades de un módulo s/E
- Oculta prop del disp o lo CPU: temporizados, formatos, electromagnéticos
- Adapte de 1 o + disp.
- Controlar o no las fx del disp:
  - Simples de E/S o procesados de E/S (manejo de parte importante de la carga de pcs).
  - Controladas de E/S o controladas de disp. (manejo primitivo)

Scanned with CamScanner

github.com/ssofiaavila

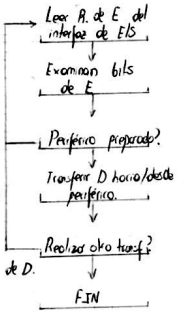
Operaciones de E/S: requieren 1- Direccionamiento: E/S mapeada en memoria - E/S aislada.
- Transferencia de inf: lectura o escritura
- Gestión de la transf: mecanismos de sincronizar y control de la transf de D.

1) Direccionamiento de E/S
- E/S asignada en memoria (memory-mapped): los disp E/S y memoria comparten un único espacio de direcciones, E/S se parece a la memoria de L/E. No hay órdenes específicas p/ E/S.
- E/S aislada: espacios de direc separados, necesidad de líneas especiales de E/S y de memoria. Órdenes específicas p/ E/S.

Técnicas de gestión de E/S: - E/S programada c/ espera de ciclo.
- E/S c/ interrupciones.
- E/S con acceso direct. a memoria (DMA)

E/S programada: es un intercambio de D e/ CPU y el módulo. La CPU tiene control sobre la op. de E/S. (comprobar est E del disp - Envío de comandos de L/E - tranf de D), la CPU espera q/ el módulo E/S termine la operax. ∴ la CPU permanece ociosa durante un período.

Detalles:
- La CPU solicita la operax de E/S al módulo.
- El módulo E/S realiza la operax.
- El módulo E/S activa los bits de E del disp. direccionado y espera.
- La CPU comprueba periódico el E de esos bits, hasta que la op. fue realizada.
- En caso contrario la CPU espera y vuelve a comprobarlo + tarde



Órdenes de E/S: La CPU emite una direc (especifica el módulo)
La CPU da una orden:
- Control: indica al módulo q/ hacer.
- Test: comprueba el E del módulo y periféricos.
- L/E: transfiere D desde o hacia el disp. x el bus de D.

E/S con interrupciones: la CPU no tiene que esperar la finalizax de la tarea de E/S, puede seguir procesando. No se repite la comprobax de las E. de los módulos. El módulo envía un pedido de interrupx a la CPU cuando está listo nuevo.

Qué hace la CPU? La CPU envía una orden de lectura (El módulo E/S obtiene los D del periférico mientras la CPU realiza otro modulo).
La CPU chequea si hay pedidos de interrupciones pendientes al final de c/ ciclo de instrux (El módulo E/S emite un pedido de interrupx a la CPU).

La CPU atiende el pedido, guarda el contexto, interrumpe el proceso y realiza la gestión de la interrupx.
La CPU solicita D (El módulo E/S transfiere los D).

Cuestiones de diseños: Identificax del módulo q/ interrumpe
- Dif. líneas p/ el módulo: PC, limita nro de disp.
- Consulta software (poll o encuesta): ocurrido un pedido de interrupx, la CPU consulta a c/ módulo p/ quien fue el demandante. Es lento.
- Conexión en cadena ("daisy chain"/hard poll): la línea de reconocimiento de interrupx se conecta encadenando los módulos, la línea de pedido es compartida. Una vez enviado la confirmax x parte de la CPU, el módulo respondió colocando un VC (palabra), en el bus q/lo identifica.

La CPU emplea el VC como puntero p/ acceder a la rutina de servicio

Interrupciones múltiples: todas las líneas tienen un orden de prioridad. Las líneas c/ + prioridad pueden interrumpir a la de < prioridad.
Si hay un maestro del bus, solo él puede interrumpirlo.

Estructura de interrupciones:
El Intel 8086 tiene solo 1 línea de petix de interrupx y 1 solo confirmax
Se debería utilizar un árbitro o gestor de interrupciones externo.
Este chip tiene 8 líneas de interrupx, x lo tanto podrá manejar 8 mód. E/S.
Es útil el Interfase de Periféricos programable (PIO)
Es un módulo de E/S de propósito gral. Posee 24 líneas de E/S programable vía reg de control.
Usado p/ una variedad de periféricos.

Análisis: las op. de E/S mediante interrupciones son + efectivas q/ las programadas.
Ambas necesitan intervenx del CPU.
Vel de transf limitada CPU ocupado + tiempo durante la operax.

Ej E/S periférico lento

Proc. a 200 MHz (tiempo ciclo reloj = 5 ns); ciclos por instr (CPI=2) promedio.
Una instrux tarda 1x 5ns = 10ns => 100 MIPS

Queremos imprimir un archivo de 10 KBytes en una impresora laser de 20 pag x min

1 pag = 3000 char (1 char = 1 byte). Imp. imprime 60000 char/min. 1 KB/s.

a) E/S c/ espera de ciclo: la CPU entra en un bucle y envía un nuevo byte c/vez q/ la impresora esté preparada p/ recibirlo.
La impresora tarda 10 seg en imprimir 10 KBytes.
CPU ocupada 10 seg. CPU podría haber hecho 1000 millones de inst.

b) E/S c/ interrupciones: la impresora genera una interrupx c/ vez q/ está preparada p/ recibir un nuevo byte.
Si la gestión de interrupx (AT) tiene 10 inst.
P/ transferir 10 KBytes tenemos q/ ejecutar 10.000 AT. 100.000 instr = 0,001 seg
Osea CPU estará ocupada E/S 0,001 seg.

Conclux lento: la E/S x interrupx ocupa en 10.000 veces el tiempo q/ la CPU se ocupa de la impresora.

Scanned with CamScanner

github.com/ssofiaavila

6699 d/e - 2021
J3

Ej: E/S periférico rápido

Procesador a 200 MHz. Una instrux tarda 2x5ns = 10ns => ~100 MIps.
Queremos transferir un archivo de memoria a disco de 10 Mbytes.
El disco posee una veloc. de transf. de 10MB/s. (1 byte c/ 10^{-7} seg o 100 nanoseg.)

a) E/S espera activa: CPU entra en un bucle y envía un nuevo byte. El disco tarda 1 seg en recibir un archivo de 10 Mbytes.
CPU se ocupa 1 seg.

b) E/S con interrup.

All = 10 instrux. P/ transf. 10Mbytes, ejecutamos 10^7 la All => ejecuta 100 millones de instr. p/ obtener el periférico. => 1 seg.

Costo rápido: no mejora tiempo.

Inicio de interrupciones:

Ambos tipos de interrupx tienen en común q' se identifican c/ un nro., el identificador de interrupx, y se requiere ejecutar una subrutina asociado a ese nro.
Hoy un VC de 255 direx de mem. una x c/ subrutina.
Al recibir una interrupx c/ identificador N accede a la posix N de dicho VC, obtiene la direx de comienzo de la subrutina y ejecuta un call a la misma.
En gral., el elem N del VC, se encuentra en N×4.

C/ elem. ocupa 4 bytes, los 2 primeros almacenan la direx, los otros 2 no se usan.

Interrupciones x hardware: INT0: Almacena el ID de interrupx asociado al PIO
INT1: " timer
INT2: " Handshake.
INT3: " COMA.

Registro AL es el único p/ instx in y out.

INT1/timer: 2 registros: CONT y COMP, direx en 10h y 11h.

CONT se incrementa automáticam' una vez p/ contar el tiempo transcurrido.
COMP contiene el tiempo límite del timer. Cuando CONT vale igual COMP, se interrumpe.

Cont. se debe reiniciar cuando se produce una interrupx.

{ IMA: registro de configurax
ISA y IMR: registros de E. }

Práctica 8

PIO interfaz de E/S que representa 2 puertos paralelos de 8 bits denominados A y B
Posee 4 reg. internos de 8 bits.
DOS de datos: PA y PB, contienen el D en los puertos A y B.
" control: CA y CB, permiten programar individual" los bits de los puertos A y B.

Conexión a partir de 30H. PA y CA: llaves (out), PB, CB: luces (out)

1) a)

b)

c)

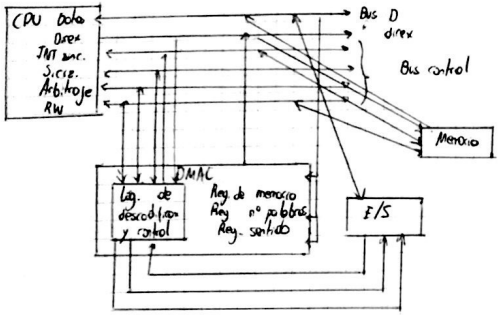
d)

Scanned with CamScanner

github.com/ssofiaavila

Clase 3 part. 2

El controlador DMA (Acceso directo a memoria) es un disp q controla una transf. de D e/ periférico y memoria sin intervenir la CPU
Debe actuar como maestro del bus en la transf. y debe ser capaz de:
- Solicitar el uso del bus c/ señales y lógica de arbitraje
- Especificar la direx de mem sobre la que se realiza la transf.
- Generar señales de control del bus. (tipo genéricas, señales de sincronización)



Etapas de una transf. DMA:

Inicialización de la transferencia
La CPU debe enviar al interfaz del periférico y al DMAC los parámetros de la transferencia
- Inicialización del interfaz (Bus master: CPU-Bus slave: interfaz): nº bytes a transf, tipo transf (L/E), otra info. (pista, sector, etc).
- Inicialización controlador DMA (" : CPU-bus slave: DMAC): nº bytes, tipo transf., direx de memoria inicial p/ transf, nº canal (p/ DMAs c/ <> canales).

Después de la inicialización, la CPU retoma a sus tareas y ya no se ocupa de la evolución de la tarea

Realización de la transferencia
Cuando el periférico está listo p/ realizar la transf, lo indica al DMAC. Pide el control del bus y realiza la transf. e/ el periférico y la memoria.

Bus master: DMAC + periférico. Bus slave: memoria.
Dsp de la transf. de c/ palabra, se actualizan los reg. del DMAC: nº bytes, direx mem.

Finalización de la transf:
El DMAC libera el bus y devuelve el control al CPU. DMAC suele activar una señal de interrupx p/ avisar al CPU la finalización de la op E/S

Problemas q puede haber:
Se puede degradar el rendimiento del CPU si el DMAC hace uso intensivo del bus.
(Si el bus está ocupado en una transferencia DMA, el CPU no puede acceder a memoria)
El problema se reduce c/ el uso de memoria caché: la mayor parte del tiempo, la CPU lee instrux de la caché, y lo q no necesita usa el bus.
El DMAC puede aprovechar estos intervalos en los q el CPU usa el caché p/ realizar las transferencias

En caso de una M sin caché, el procesador no utiliza el bus en todas las fases de la ejecux de una inst.
El DMAC aprovecha cuando el CPU no utiliza la memoria

Tipos de transferencia:
Si el DMAC sólo toma el control del bus durante los intervalos de tiempo en los q el CPU no hace uso del mismo, el rendimiento del sist. no sufrirá degradación alguna.

Hay 2 tipos de transferencia: ráfaga (burst) o por robo del ciclo (cycle stealing)

DMA modo ráfaga:
El DMA solicita el control del bus a la CPU. Cuando la CPU concede el bus, el DMAC no lo libera hasta haber finalizado la transferencia de todo el bloque de D completo.
ventaja: transf rápida
desventaja: durante el tiempo q dura la transf. la CPU no puede usar el bus de mem y puede degradar el rendimiento del sist.

DMA modo robo de ciclo:
DMAC solicita control al CPU. Cuando lo concede, se hace la transf de una única palabra y DMAC libera el bus.
DMAC solicita bus las veces q sea necesario hasta finalizar de transferir el bloque de D.
ventaja: no hay degradación
desventaja: transf + lenta.

P/ la CPU no es una interrupx (pues no guarda contexto). Si bien el flujo es lento, no será tanto como si ella realizara la transf ∴ p/ transf de E/S de múltiple palabra, es lo + eficiente.

módulo controladores de periféricos.

Canales de E/S
Los disp de E/S son cada vez + sofisticados. Al principio, CPU controlaba periféricos. Se agregó el módulo E/S o controlador. Luego 2 + llamado de interrupx. a) El módulo E/S incluye DMA. b) módulo E/S tiene su propio procesador c/ su pequeño conj de instrux. c) El módulo además tiene mem local, o sea se convierte en una comp en sí misma.

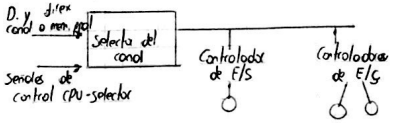
Caract. de canales E/S
- Representan una extensión del concepto del DMA (tienen habilidad de ejecutar instrux).
- Completo control de la transf de D ∴ la CPU no ejecuta inst E/S.
- Programa almacenado en mem ppal.
- La CPU inicia la transf E/S (ordena ejecutar el programa q está en mem - El program. especifica dispositivo, áreas de memoria a usar, prioridades y acciones ante errores.

Scanned with CamScanner

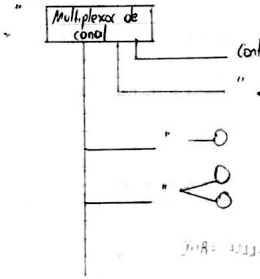
github.com/ssofiaavila

Tipos de canales de E/S

Selector:
Controla varios disp. de ↑ velocidad y ↓ nº, ∴ el canal se dedica pl lo transf. de D de ese disp.
El canal selecciona un dispositivo y efectúa la transferencia. Los disp. son manejados x un controlador o módulo E/S. ∴ el canal E/S ocupa el lugar de la CPU en el control de esos controladores.



Multiplexor
Puede manejar E/S con varios disp. a la vez.
Multiplexor de bytes: intercala y admite caracteres.
" de bloques: intercala bloques de D desde <> disp.



P.3
1)e)

M ocupada
80x + 4 + 100 = 404
M disponible = 100 + 4 = 104
Tiempo = 3+2·8+3 = 8

Handshake

Handshake: intermediario el CPU y la impresora

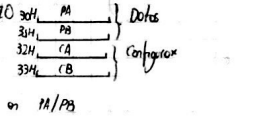
18544/3

P.3
1)e)

```
FOR EQU 20h        ORG 1000H
IMR EQU 21h        MI BB 0,1,2,3,4,5,6,7
INT EQU 25h        FIN BB ?
CONT EQU 10h       ORG 2000h
COMP EQU 11h       cli
PB EQU 31h         mov AL,TD.timer
CB EQU 33h         out INT,AL
TD.TIMER EQU 10    mov AL,11111036
org 40             out IMR,AL
                   mov AL,1
ORG 3000H          out COMP,AL
imprimir_mio:      mov SI,0
   CMP CL,7        cmp CONT,AL
   JNZ ocupada     mov AL,0
   JMP volver      out CB,AL
ocupada: mov AL,[BX+SI]   mov BX,offset MI
   out PB,AL       mov CL,offset fin - offset MI
   INC BX          sti
   INC CL       loop: jmp loop
   JMP ocupada     END
volver: mov AL,[BX+SI]
   out PB,AL
   INC BX
   INC CL
   JMP volver
```

Entrada/Salida
Gran variedad de periféricos c/ varios métodos de operar, pero todos son + lentos q el CPU y RAM.
Necesidad de una interface c/ el procesador/la memoria y los periféricos.

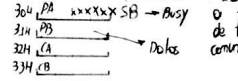
NS×88 usa E/S aislada.



- Un bit en 0 en CA/CB selecciona como S o la línea correspondiente en PA/PB
- " 1 selecciona como E o la línea correspondiente en PA/PB

Luces/Llaves
Encendido/Apagado de las luces (periférico de salida) mediante la basura de microcontroladores o llaves (perif. de E), ambas comunicadas c/el microprocesador a través de puertos paralelos del PIO.

Impresora



Escribir un programa q envíen D a/a impresora a través de la PIO. La PIO debe cumplir los ↑ de temporización q requiere la impresora pl la comunicación.

- Luces: si no configuro CB, los D de PB serán ignorados.

Scanned with CamScanner

github.com/ssofiaavila

Ejemplo - PIO.mp

```
PIO EQU 30H
  ORG 1000H
MSJ DB "ORG. y ARQ. DE comp"
FIN DB ?
  ORG 2000H
  mov AL, 0FDH   } CA = 1111 1101 → B: E  S: S.
  out PIO+2, AL  }
  MOV AL,0       } CB = 0000 0000 → PB = Salida
  out PIO+3, AL  }
  IN AL,PIO      } Fuerzo Strobe a 0
  AND AL,0FDH    }
  OUT PIO, AL    }
  MOV BX, offset msj  } Inicializo BX y CL
  mov AL, offset fin - offset msj }
poll: in AL,PIO  } Leo E. de Busy y no avanzo hasta q
  AND AL,1       } este libre (0)
  JNZ POLL       }
  MOV AL, [BX]   } Cobro char a imprimir en el
  OUT PIO+1, AL  } PB
  IN AL,PIO      } Fuerzo strobe a 1
  OR AL, 02H     }
  out PIO, AL    }
  IN AL,PIO      } Fuerzo strobe a 0
  AND AL, 0FDH   }
  out PIO, AL    }
  INC BX         } Apunto al sig char
  DEC BCL        }
  JNZ POLL       }
  INT 0
END
```

Impresora c/ PIO

PA, 30H: xxxxxxSB — R. E → R.O
PB, 31H:
CA, 32H: xxxxxx01 → configuro S y B
CB, 33H: 00000000
configuro 'salida' de PB: Strobe bit S, Busy bit E

Práctica 3

2) a)

```
PA EQU 30H
PB EQU 31H
CA EQU 32H
  ORG 1000H
letra DB 'A'
  ORG 3000H
strobeC: IN AL,PA
  AND AL,0FDH   1111 1101b
  OUT PA, AL
  RET
  ORG 2000H
strobe1: IN AL,PA
  OR AL,02H
  OUT PA, AL
  RET
  ORG 2000H
  mov AL,0FDH   1111 1101
  OUT CA,AL
  mov AL,0H
  out CB,AL
poll: in AL,PA
  AND AL,1
  JNZ poll
  mov bx, of a otra
  mov CL,[BX]
  out PB,AL
  call strobeC
  call strobe1
  INT 0
  HLT
```

c)

```
  ORG 1000H
CHAR DB ?
  org 2000H
  mov CL, 5
  ...
  mov AL, 01H
  out CA, AL
  mov AL, 0H
  out CB, AL
inicio: CMP CL, 5
  JZ fin
poll: IN AL, PA
  AND AL,1
  JNZ poll
  call strobeC
  mov BX, offset char
  INT 6
  call strobe0
  MOV AL, [BX]
  out PB, AL
  call strobe1
  DEC CL
  JMP poll
fin: INT0
  END
```

d)

```
FOR EQU 20H
  NL_FIJO EJO 10
  ORG 40
DB out_FIJO
  ORG 1000H
char DB ?
  ORG 2000H
  sti
  mov AL, 11111010b
  out PIC+1, AL
  mov AL, N FIJO
  out PIC+4, AL
  mov AL, 0FDH
  out CA, AL
  mov AL, 0H
  out CB, AL
  sti
inicio: mov BX, offset coac
  INT 6
  INC BX  INC CL
  JMP bucle
  INT 0
```

```
3000H
out_FIJO: poll: IN AL, PA
  AND AL, 1
  JNZ poll
  call strobeC
  mov BX, offset char
  mov AL, [BX]
  out PB, AL
  strobe 1
  DEC CL
  JZ fin
  JMP poll
fin: mov AL, 20
  out EOI, AL
```

Clase 4: Segmentos de instrucciones

Segmentos de cauce concepto básico

La segmentación de cauce (pipelining) es una forma efectiva de organizar el hardware de la CPU p/ realizar + de una operación al mismo tiempo. Consiste en descomponer el proceso de ejecución de las instrucciones en fases/etapas q permiten una ejecución simultánea.

Explota el paralelismo e/ las instrucciones de un flujo secuencial

Ejemplos de estructuras

Similar a la línea de armado en una planta de alguna manufactura. El auto pasa a varias E en el proceso de producción / varios productos pueden ser manipulados simultáneamente (uno en E c/u).
Se puede comenzar el proceso nuevo (E o la línea de prod.) antes de q salga el prod final de la misma.

Características (de todos estos tipos)

- La segmentación es una técnica de mejora de prestaciones a nivel de diseño hardware
- La segmentación es invisible al programador.
- Necesidad de uniformizar las etapas: el tiempo de la + lenta.
- El diseño de procesadores segmentados tiene gran dependencia del repertorio de inst

Scanned with CamScanner

github.com/ssofiaavila

Tareas a realizar x cada

Búsqueda (F, fetch): se accede a memoria x la instrux, se incrementa PC.

Decodificación (D, Decode): se decodifica la instrux, obteniendo operax a realizar en la cual de D. Se accede al banco de registros x el/los operando/s (y es necesario). Se calcula el valor del operando inmediato (x extensión de signo (si hace falta). Banco de registros

Ejecución (X, Execute): se ejecuta la operación en la ALU ALU

Acceso a memoria (M, Memory Access): si se requiere un acceso a memoria, se accede. Memoria de D

Almacenamiento (W, write back): si se requiere volcar un dato o un registro, se accede Banco de registros al banco de registros.

Repertorio sencillo de instrucciones

Instrucción	Pseudocódigo	Descripción
LW	LW RT, inmed(RS)	Carga reg RT desde memoria
SW	SW RT, "	Almacena en memoria desde reg RT
ADD	ADD RD, RS, RT	Suma palabra en reg RS y RT en RD resto en RD
SUB	SUB RD, RS, RT	Resta
AND	AND RD, RS, RT	AND palabra RS y RT, resto RD
OR	OR RD, RS, RT	OR "
SLT	SLT RD, RS, RT	Pone 1 en RD si RS es <= q RT
BEQ	BEQ RS, RT, destino	Salto a "destino" si RS es = RT

Formato de instrux

Tipo I: | OPCODE 6 bits | RS 5 bits | RT 5 bits | Inmediato 16 bit | } 32 bits

Tipo R: | OPCODE 6 bits | RS 5 bits | RT 5 bits | RD 5 bits | SHAMT 5 bits | FUNCT 6 bits | = 32 bits

Tipo J: | OPCODE 6 bit | Offset sumado al PC 26 bit |

Prestaciones del cauce segmentado.

Teórico: el máx rendimiento es completar una instrux c/ c/ ciclo de reloj.
Si K es el nro. de etapas del cauce => rel. procesador segmentado = rel. secuencial x K

El incremento potencial de la segmentación del cauce es proporcional al nro. de etapas del cauce.

Incremento la productividad (throughput), pero no reduce el tiempo de ejecución de la instrux.

Análisis de la segmentación
Suposiciones: todas las tareas duran = tiempo. Las instrucciones siempre pasan x todas las etapas. Todas las etapas pueden ser manejadas en paralelo.

Problemas: no todas las instrucciones necesitan todas las etapas.
SW RT, inmed(RS); no uso W (En msx88; un mov Ax, mem)

No todas las etapas pueden ser manejadas en paralelo.
F y M acceden a memoria.
No se tienen en cta. los saltos de control

Riesgos estructurales: 2 o + instrucciones necesitan usar el mismo HW hardware en el mismo ciclo

Tipos de dependencia de D

Lectura disp de escritura (RAW, dependencia verdadero): una instrux genera un D q lee otra posterior.

Escritura disp de Escritura (WAW, dependencia en S): una instrux escribe un D disp de otra posterior. Sólo se da si se deja q las instrucciones se adelanten unas o otras.

Escritura disp de lectura (WAR, ant. dependencia): una instrux modifica un valor antes de q otra anterior q lo tiene q leer, lo lee. No se puede dar en nuestro cause simple.

Cap. 11, Wstalling - Cap. 1 - Beltrán

Handshake - 2 registros: No registros configurables Dato 4bh DDDDDDDD E 4bh xxxxxxSB 26h→INT2 PIO

Debe hacer consulta de E

```
Imprimir un caracter
dato  EQU 4bh
estado EQU 4Ah
      ldoch
char db "a"
      org 2coch
loop: in al,estado
      and al,1
      jnz loop
      mov al,char
      out dato,al
      int 0
      end.
```

```
Imprimir cadena
      org 1coch
str   db "Mi cadena"
      org 2coch
      mov cl,9
      mov Bx,offset str
loop: in al,estado
      and al,1
      jnz loop
      mov al,[Bx]
      out dato,al
      inc bx
      dec cl
      jnz loop
      int 0ord.
```

```
;interrupx → a el bit + significativo
Busy=0 / envío interrupx al PIC

El d. se imprime c/una subrutina de interrupx
                              Estado EQU 4Ah
;configuración hexa
      org 1coch
      in al,Estado
      or al,1coch : 1000 0000
      out estado,al
; al PIC. →
```

Scanned with CamScanner

github.com/ssofiaavila

Práctica 3

3) a)

b) No hace falta configurar pio y se libera automáticamente la impresora.

c)

*configura: imr, int2, ic interrup.

```
IMR EQU 21h
INT2 EQU 26h
org 16
dir_imp dw 3000h

org 4000h

mov al, 0FBh; = 1111 1011
out IMR, al
mov al, 4
out INT2, al
```

programa completo. 1 caracter

```
EOI EQU 20h
IMR EQU 21h
INT2 EQU 26h
DATO EQU 40h
ES EQU 41h
org 3000h
imp: mov al, char
out dato, al
mov al, 0FFh
out IMR, al
mov al, 20h
out EOI, al
iret
org 2000h
ct:
..
..configurar PIC y PIO
loop: jmp loop
int 0
hlt
end
```

or al, 80h → 3000 0000 { P/el bit + significativo
out Estado, al

*Transferencia DMA: ordena como maestro: arbitra bus. Solicita uso del bus c/ señales.
Transf: ráfaga y robo de ciclo.
Ráfaga no lo libera hasta terminar. rapido - degrada rendimiento
Robo de ciclo: no degrada + tiempo

UART

*configurar

cch = DIN → D o leer del disp
61h = DOUT → D o enviar al disp.
62 h = CONTROL → configurar (escritura) y lectura del E (p/ polling)

Protocolo 4) 0=XON/XOFF 1=DTR s/encriptado

7, 6, 5, 4, 3, 2, 1, 0 → siempre encendido
sincronismo
Error reset 6) 0=deshabilitado 1=habilitado
velocidad
1 siempre

62h estado 7, 6, 5, 4, 3, 2, 1, 0
0) Tx ready = si está listo p/q mandemos 1 siempre
1) Rx ready si hay algo p/ leer
7) 0= ocupado dice si 1=libre impresora está ocupado
0= buffer lleno 1= disponible

DTR= polling de handshake. Loop hasta que usart envíe Tx ready y disp loop hasta q el disp dice RX

XON/XOFF: si hay char p/ imprimir: 1) espera Tx ready (bit - significativo. 2) Envía char a DOUT 3) consulta Rx ready si hay D p/ leer. → si lo hay, lee y si es XOFF, espera Rx Ready otra vez hasta leer XON y seguimos.

50h: puerto de origen parte baja
51h: " alto
52h: cant bytes a transferir - bajo
53h: cant " alto
54h: dir destino - bajo
55h: " - alto
56h: control
57h: arranque - escribo F7h

AISC deben ser op simples como: aritmética-lógicas, instr de flujo (read, write, leer e imprimir)

q' a lo sea pueden tener <> modif respectivo a PC si se dirige de salto / condicional c/ registro si no lo se)

Instrux pueden ser I/O variable c/ pipe o híbrido

Procesador monociclo → cd instrux se ejecuta en un único ciclo de reloj
multiciclo → " en varios ciclos

Scanned with CamScanner

github.com/ssofiaavila

MIPS - procesador tipo RISC - conjunto reducido de instrucciones.
Instrux 32 bits, registros de 64 bits, 16 memoria programa (2^{30} palabras), no hay flags.

Al haber una memoria de programa y otra de D, hay buses independientes.
formato instr no depende formato mem. de D.
32 registros de uso gral. 32 reg. P. flotante. Se usan registros a vez de flags,
R0 siempre es 0 no cambia. de f0 a f31. no hay pila

- .code o .text indica q' el cód q' sige es cód fuente
- .data: indica q' lo q' sige son D.
- .org p' direc específica
- .space: reserva zona de memoria

Datos: .ascii {string}: ej: mi_variable: .ascii "hola"; 4 bytes.
.asciiz: almacena cadena terminada en 0. mi_variable: .asciiz "hola"; 5 bytes.

- .byte: almacena 1 byte. mi_variable .byte 123; 1 byte.
- .word 16: almacena nros de 16-bits. mi_variable .word16 1234; 2 bytes
- .word32: almacena nros de 32 bits. m_variable .word 1234; 4 byte
- .word: " 64 bits. mi variable .word 1234; 8 byts.

- .double: pto. flotante. de 8 bytes
- .halt: detiene procesador
- .nop: de no hace nada

Tipos instrucciones: · Aritmético lógicos (sólo c/ registros)
op <reg_destino>, <reg1>, <reg2> / op <reg_dest>, <reg_1>, <inmediato>

· Acceso a memoria (el reg y mem)
sd <reg>, <dirrex>
id <reg>, <dirrex>

· Saltos j <etiqueta>
jal <etiqueta. subrutina>
beq <registro>, <etiqueta>
- beq <reg1>, <reg2>, <etiqueta>

Práctica 4

1) a) daddi r2, r0, 10 — mov r1, r2
b) daddi r5, r5, 5 — mov r5, r5
c) sub r4, r4, r4 — mov r4, r4
d) cero r5, -1
e) xor r6, r6, 0xffff — not

2) a) Genera otro sd r2, M(10) porque aun no se escribió el valor q' solicita p' leer. Solo tipo RAW/dependencia de D si pasando es 2 200
b) Pipe se evita leyendo CPI es 3800

3) a) Lazo depende de D
Branch taken stalls
1750 CPI, 25 ciclos, 12 instrucs
b) Ciclo otro daddi r1, r1, 1
y beqz r2, loop
Data 3 ciclos
Se evita el RAW, el otro otro sigue
Tiene 2250 CPI, 4 inst q antes

CPI = ciclo / instrucciones → ideal, no real de libro
ideal → sin riesgos CPI: 5n/N=5 (secuencial)
5 → hay 5 etapas de ej ciclo

forwarding: no se necesita stall
no se necesita WB porque ...

Parcial 1er cuat

1) Leer char desde teclado, mostrar en pantalla c/ subrutina IMPRIMIR, char x referencia

```
org 3000H
mov AL,1
INT 7
RET
```

```
ORG 2000H
char DB ?
fin DB ?
ORG 2001H
mov BX,offset char
INT 6
call imprimir
INT 0
END
```

2)
```
EOI  EQU 20h
PB   EQU 31h
CB   EQU 33h
IMR  EQU 21h
INTD EQU 24h
E_IO EQU 20
ORG 2000H
cli
mov AL,0FEh
out IMR,AL
mov AL,E_IO
out INTA,AL
mov CL,00h
out CB,AL
sti
loop: jmp loop
INT 0
END
```

```
org 40
DIR: int_E_IO
ORG 3000H
int_E_IO:
mov AL,0FFh
out PB,AL
mov AL,20h
out EOI,AL
IRET
```

Branch taken stalls (BTS): otros x toma equivocada ej halt en wbe

3)
```
PB EQU 31h
CB EQU 33h
org 2000h
mov AL,00h
out CB,AL
mov AL,0FAh
out PB,AL
INT 0
FLD
```

```
org 3000h
push BX
mov BX,SP+2
mov AL,1
INT 7
pop BX
IRET
```

Práctica 4 stalls
Dependencias en instrucciones. 3 tipos

Dependencia de D: inst× necesita acceder a un reg pendiente de lectura o escritura × otra inst
· de salto: una instrux de salto determina q' ∫o sig. ptl se case no debe ejecutar
· estructural: las instrux intentan acceder a un mismo HW

Etapas

IF (instruction fetch): obtiene la sig instrux de dirección de PC
ID decodifica instrux busca operandos
Ex: ejecuta inst.
M: memory access: accede a la memoria requerida x la instrux actual
WB: escribe result. en el archivo

4) a)
b)

	Nro ciclos	CPI	RAWs	Branch taken stalls
Forwarding	71	1869	16	0
Branch taken	75	1704	2	4

Scanned with CamScanner

github.com/ssofiaavila

Branch target Buffer

Branch Misprediction stall (Ahorro x pred x corrección) ciclos

Delay Slot

5) a) Ejecutar el loop completo no tiene predicción

b) Se ejecuta el generico un promedio de 1933 CPI

c) 22 ciclos, 12 instrucciones

6) .data
num1: .word 3
num2 .word 8
RES: .word 0
.code
ld R1, NUM1(r0)
ld R2, NUM2(r0)
daddi R3, R0, R0
loop: dadd R3, R3, R1
daddi R2, R2, -1
bnez R2, loop
sd R3, RES(r0)
halt

7) a) .data
notas: .word 1, 8, 0, 5, 4, 7, 11, 8, 3, 2
X: .word 5
RES: 0
CANT: 0 ANDO: .word 0x8000
.code
lb r7, ANDO(r0)
lb r2, X(r0)
lb r4, RES(r0)
lb r6, CANT(r0)
xor r5, r0, r0
loop: ld r1, notas(r5)
and r8, r1, r2
beqz r8, epar
daddi r6, r6, -1
lb r2, 1 (r0) ; r2=1 RE
daddi r5, r5, 8
epar: daddi r4, r4, -1
; mover 1 a RE
and r8, r1, r7
ld r3, r2(r3)
ld r3, r3(r3)
bnez r3, loop

halt

9) .data
A .word 4
X .word 0
Y .word 2
.code
ld r1, A(r0)
ld r2, X(r0)
ld r3, Y(r0)
loop: daddi r2, r2, r3
daddi r1, r1, -1
bnez r1, loop
sd r2, X(r0)
halt

Un reg genero un resultado p/ q otro reg operativo

Clase 5 - Posibles soluciones a atascos

Atasco de un cause:
Situaciones q' impiden a lo sig. instrux q' se ejecute en el ciclo q' le corresponde

- Estructurales: provocados x conflictos de RR (hardware)
- Dependencia de D: 2 instrux se comunican x medio de un salto.
- " de control: la ejecux de una instrux depende de cómo se ejecute otra

Si resolvemos c/ paradas del cauce, disminuye el rendimiento teórico y CPI > 1

Soluciones a riesgos estructurales:

- Simple: replicar, segmentar ó realizar turnos p/ el acceso a los unidades forzosas en conflicto.
  - Duplicar ∫ de RH de hardware, sumadores o restadores a parte de ALU
  - Separar en memorias de instr y D. → 2 memorias
  - Turnar el acceso al banco de registros: escrituras en la 1ª mitad de los ciclos de reloj / lecturas en la 2ª mitad de los ciclos de reloj. En un mismo ciclo accede p/ <> casos.

Soluciones a riesgos de D:

- P/ riesgos de RAW: se debe determinar cómo y cuando aparecen estos riesgos. Será necesario una unidad de detenx de riesgos y/o compilador + complejo.

2 soluciones
Hardware: adelantamiento de operandos (forwarding) → Es una retroalimentación de ALU
Software: instrux NOP o reordenox de cód.

Técnica de hardware/ adelantamiento o forwarding): consiste a pasar directo el dato obtenido c/ una instrux a las instrux q' lo necesiten como operando.
Si el D necesario está disponible a la salida de la ALU (X_i) se lleva a la E. de la etapa correspondiente (X_{i+1}) sin esperar a la escritura (M_i o W_i). Ocurre q' los datos no deben esperar a q' se escriba en el reg de destino
Fácil de implementar si se identifican todos los adelantamientos y se comunican a los registros de segmentox correspondientes.

Técnica de software: evita los riesgos reordenando las instrux del cód. sin afectar datos.
Es realizada x el compilador: introdux del NOP (espera retardo) - reordenox de instrux (máx separox de instrux c/ dependencia RAW, cuidado c/ ejecux fuera de orden).

Soluciones a riesgos de control: existe penalizax x salto → tiempo perdido
Instrucciones de salto:

- Incondicional: la direx de dest. se debe determinar lo + pronto posible, dentro del cause, p/ reducir penalizax
- Condicional: introduce riesgo condicional adicional x la dependencia c/ la condix de salto y el dato de una instrux previa

Modificax sencilla de la ruta de D p/ reducir la cant de paradas a un solo ciclo.

Adelantar la resolux de los saltos a la etapa D: en ella se decodifica y se sabe q' es un salto. Se puede evaluar la cond. de salto (c/ restador), se puede calcular la direx de salto (c/ sumador)

P/ tratamientos de saltos hay:

- Técnica de hardware: predix de saltos p/ evitar la parada
- Software: salto retardado o de relleno de ranura de retardo. El compilador introduce instrux q' se ejecutarán en cualquier caso dep. de la instrux de salto.

Scanned with CamScanner

github.com/ssofiaavila

Técnicas dinámicas: -computadas saltar/no saltar: basado en la historia de las instrux
ofrecen p/ bucles.
- Tabla de historia de saltos (Branch-Taken-target buffer) Es una pequeña caché asociada a la etapa de búsqueda (F) q' tiene 3 campos · Direc de instrx de bifurcax
· Info instrux dest →
· N bits de E

Salto retardado: realizar trabajo útil mientras el salto se resuelve.

Hueco o ranura de retardo de salto (delay slot) es el periodo de parada o parada luego de una instrux de salto.
El compilador trata de situar instrux útiles (q' no dependan del salto) en los huecos de retardo. Si no es posible, se usan instrux NOP.
Las instrux en los huecos de retardo de salto se captan siempre.

Requiere reordenar instrux.

*Otras soluciones de hardware: -Predecir s/ el código de operax: hay instrux c/ + probabilidad de saltar. La tasa de acierto puede llegar al 75%.
- Buffer de bucles 2)
- Flujos múltiples 1)
- Precaptar el destino del salto = precapto

1) Flujos múltiples: varios cauces (uno x c/ opx de salto), precaptan c/salto en <> saltos.
Se debe utilizar el cauce correcto.
Desventajas: pruvna retardos en el acceso al bus y registros. Si hay múltiples saltos, se necesita un > nro. de cauces.

·Precaptar del destino de salto: se precapta la instrux destino del salto, además de las instrux sig a la bifurcax.
La instrux se guarda hasta q' se ejecute la instrux de bifurcax.

2) Buffer de bucles: memoria muy rápida. Gestionada x la etapa de captax de instrux del cauce.
Comprueba el buffer antes de hacer la captax de memoria.
Muy eficaz p/ pequeños bucles y saltos.

cap 11 - Stallings - Cap 1 Beltrán, Guzmán

Práctica 5 - explicar

Hasta ahora se utilizaron nº 32, ahora se usan de pto. flotante, lo cual incrementaría los CPI. Se encontró la solux de incluir nuevos componentes como un sumador, divisor y multiplicador. Permitirá ejecutar instrux en < tiempo pero surgirán nuevos problemas. stalls estructural / WAR / WAW provocados x conflicts de RR.

Sólo sucede cuando 2 instrux en EX acceden a la etapa de memoria simultáneo, tiene prioridad lo 1º q' entró a pipeline.

WAW y WAR stalls: suceden cuando hay + de una unidad de ejecux, dependencia de D of 2 instrux. Una instrux q' entra al cause puede sobrepasar una instrux anterior.
El simulador genera estos stalls cuando detecta situax potencial de conflicto

Subrutinas + reducidas q' x86. No hay manejo implícito de la pila (CALL y RET), no hay instrux explícitas p/ PUSH y POP

Problemas: - Registro de retorno: si el prog. ppal. invoca a una subrutina y dentro de esa subrutina se invoca otra, el 2º jal sobreescribirá el valor de $ra c/la nueva direc. de retorno.

·Sobreescritura de reg: si una subrutina usa registros de la subrutina q' la invocó, puede suceder q' los valores del mismo no se conserven al retornar

P/ resolver estos problemas, se estableció una convenx: -Fx particular p/ c/ registro
·Mecanismo p/ invocar subrutinas
·Implementax de pila

Registros, nombres y usos.

$zero	siempre vale 0, no puede cambiarlo	$0
$ra	Retun address - Dir. retorno de rutina - Debe ser salvado	$31
$v0 - $v1	Val. de retorno de la subrutina llamada	$2-$3
$a0 - $a3	Argumentos pasados a la sub. llamada	$4-$7
$t0 - $t4	Reg. temporales	$8-$15 y ($24-$25)
$s0 - $s7	q' deben ser salvados	$16-$23
$sp	Stack pointer - Puntero tope de pila - Debe ser salvado	$29
$fp	Frame pointer - puntero de pila - Debe ser salvado	$30
$gp	P/ uso del Kernel del SO	$28
$at	Asembly temporary - p/ ensamblador	$1
$k0 - $k1		$26-$27
$sp	global pointer: puntero zona mem. estática - Debe ser salvado	$28

Ejemplo

```
.data
result: word 0
.text
    daddi  r4, r0, 10
    daddi  r5, r0, 20
    jal    suma
    sd     r2, result(r0)
    halt
suma: daddi r2, r4, r5
      jr    $ra
```

→

```
.data
result: word 0
.text
    daddi  $a0, $zero, 10
    daddi  $a1, $zero, 20
    jal    suma
    sd     $v0, result($zero)
    halt
suma: daddi $v0, $a0, $a1
      jr    $ra
```

Preservax de registros

Una subrutina debe garantizar q' preserva los val originales de reg.

Así, una subrutina puede llamar a otra subrutina sabiendo q' ésta no modificará el val. de regs.

Es necesario contar c/ un lugar p/ almacenar los val. originales de los reg

Se puede hacer en la mem ppal c/ una pila

Convenx uso de pila: al no haber instr. específicas, x convenx todos los subrutinas usarán el reg. $sp como puntero al tope de pila.

push $t1 → daddi $sp, $sp, -8
sd $t1, 0($sp)

pop $t1 → ld $t1, 0($sp)
daddi $sp, $sp, 8.

Scanned with CamScanner

github.com/ssofiaavila

Punto flotante tengo 32 registros de $f0 a $f32.

Ej calcular suma de vol. del vec multiplicar por 0.5 Si es > 10 multiplico por 2. Guardo en RES.

```
.data
vec:   .word 1,4,2,6
vol0:  .double 0.5
vol10: .double 10
vol2:  .double 2
res:   .double 0

.code
daddi  r4,r0,0
daddi  r2,r0,4   ;#elem
l.d    f1, res(r0); #suma
l.d    f2, vol0(r0); 0.5
loop:
    ld   r3,vec(r1)
    mtc1 r3,f3   ; convierto a flotante y multiplico x 0.5
    cvt.d.l f3,f3 ; y sumo
    mul.d f3, f4, f2   ; multiplico x 0.5
    add.d f1, f1, f3   ; agrego a reg de suma
    daddi r1, r1, 8
    daddi r2, r2, -1
    bnez  r2, loop

// si es mayor a 10

    l.d  f4, vol10(r0)
    c.lt.d f2, f4        ; res de sacarlo a f0
    bc1f fin
    l.d  f5, vol2(r0)
    mul.d f1, f1, f5
fin: s.d f1, res(r0)
    halt
```

Pipeline segmentado

Práctica 5

1) a) 16 ciclos, 7 instrucciones, 2.286 CPI.

b) Se generan 3 dependencias de D. Las dobles en conflicto son f2, f3, f4 en las instrucciones de s.d, y add.d.

c) Aparecen ya 2 o + instrucciones quieren acceder al mismo RR. aparece en los 2 ciclos s.d. En la etapa de EX.

d) Porque lo quiere leer antes que se haya terminado de llegar escribir en la etapa anterior.

e) Va a ir al retraso aparato la etapa en mem. del UOP

2) mtc1: copia 64 bits de un reg de enteros a uno flotante
cvt.l.d: convierte 7L a pto flotante

cvt.d.l: pto flotante a 7L

mfc1: copia bits de FP a reg 7L

3) .data
vec: .double 5.85
cero: .double 0.47
vp: .double 0.0
.code
l.d f0, vec(R0)
f2, cinco(R0)
mul.d f3, f0, f2
daddi, r2, r0, 2
mtc1 r2, f4
cvt.d.l f4, f4
daddi r1, r0, 4
cvt.d.l f1, f5, f4
mul.d f6, f3, f2
div.d f5, f1, f6

4) .data
pero: .double 75.7
cinco: .double 1.73
estado: .asciiz "estado"
.code
ld f0, pero(R0)
ld f2, cinco(R0)
daddi, r20, r0, 2
mtc1 r2, f4
cvt.d.l f4, f4
mul.d f6, f0, f2
div.d f5, f1, f4

5) $zero vale 0
$at kernel del SO
$v0-$v1 valores de retorno
$a0-$a3 parámetros p/ a función
$t0-$t7 reg temporales
$s0-$s7 deben ser salvados ✓
$t8-$t9
$k0-$k1 p/ ensamblador
$gp global pointer ✓
$sp puntero ✓
$fp frame pointer
$ra return address ✓

6) a) El programa calcula la potencia 16^4. Está estructurado en un programa ppal y una subrutina.

b) Salto a la subrutina etiquetada, y salto a una dirección de un registro.

c) Antes de saltar a la otra subrutina debería guardar en la pila el la dirección de retorno a mi programa ppal.

Scanned with CamScanner

github.com/ssofiaavila

7)

```
.data
.word 5
.text
ld $a0, labio($zero)
ld $a1, M($zero)
daddi $a1, $zero, 7
jal compara
sd, $v0, (onde_vero)
halt
compara: daddi $t3, $zero, 0
loop: slt $t1, $a0, 0
      bnez $t1, fin
      slt $t1, $a0, $a1
      daddi $a0 $a0, 8
      bnez $t2, incrementar
      beqz $t1, loop
bnez
      daddi $t2, $t3, 1
      jr loop
j2
      daddi $v0, $zero, $t3
      jr $ra
```

8)

```
.data
cadena1: .asciiz "hola"
cadena2: .asciiz "hola"
result: .word 0.
      .code
      daddi $a0, $0, cadena1
      daddi $a1, $0, cadena2
      jal compara
      sd $v0, result ($zero)
      halt
compara: daddi $v0, $0, 10
   loop: lbu $t3, 0($a0)
         lbu $t4, 0($a1)
         beqz $a0, fin-a0
         beqz $t3, fin1
         bne $t0, $t4, fin1
         daddi $v0, $v0, 1
         daddi $a0, $a0, 1
         daddi $a1, $a1, 1
         daddi $a3, $a3, 1
         loop
fin-a0: bnez $t4, fin1
         daddi $v0, $0, -1
fin1: jr $ra
```

10)

```
factorial
      daddi $sp, $sp, -16
      sd $a0, 0($sp)
      sd $a0, 8($sp)
      beqz $a0, fin-uno
      daddi $a0, $a0 -1
      jal factorial
      daddi $a0, $a0, 1
fin:  daddi $v0, $a0, 1
      ld $a0, 8($sp)
      daddi $sp, $sp, 16
      jr $ra
```

9)

```
.data
letra: .asciiz "0"
vocales: .asciiz "AEIOUaeiou"
result: .word0
      .code
      lbu $a0, letra ($0)
      jal es_vocal
      sd $v0, result ($zero)
      halt
es_vocal: daddi $v0, $0, $0
      daddi $t0, $0, 0
loop: lbu $t1, vocales ($0)
      beqz $t1, fin_vocal
      beq $a0, $t1, si_es_voc
      daddi $t0, $t0, 1
      j loop
si_es_voc: daddi $v0, $0, 1
fin_vocal: jr $ra
```

10)

```
.data
cadena: .asciiz "abecedario"
vocales: .asciiz "AEIOUaeiou"
result: .word 0
      .code
      lbu $a3, cadena ($0)
      daddi $sp, zero, 0x400
      jal recorrido
      sd $v0, result ($zero)
recorrido: daddi $t4, $0, 1
loop:  beqz $a3, fin
       lbu $a0, cadena ($0)
       daddi $sp, $sp -8
       sd $ra, 0($sp)
       jal es_vocal
       daddi $a3, $a3, 1
       bnez $t4, loop
fin:  ld $t5, 0($sp)
      daddi $sp, $sp, 8
      daddi $ra, $0, $t5
      jr $ra
```

Clase 6 - RISC

↳ Computadoras de repertorio reducido de instrucciones

Antecedentes de evolución

Familia: <> partes c/ = arquitectura pero <> organizax
- fue introducido x IBM en System/360 en 64'
- Separa la arquitectura de la implementax

- Unidad de control microprogramada: idea de Wilkes en 1951.
- Memoria caché: en 1966.
- RAM de estado sólido: que tiene menor espacio
- Microprocesadores: comienzo con Intel 4004 en 1971. variantes: propósito gral. y embebidos/empotrados.
- Procesadores multiple.
- RISC computadoras de repertorio reducido de instrucciones.

Características ppales:
- gran cro de registros de uso gral o mejor tecnología de compiladores p/ optimizar el uso de los registros
- Repertorio de instrux limitado y sencillas.
- Enfasis en la optimizax de la segmentax de instrux
- facilitar trabajo del escritor del compilador
- Mejorar la eficiencia de ejecux: secuencias complejas de op microcódigo.
- Dar soporte a HLL + complejos

- el software es + cero que el hardware
- U.V. de lenguaje c/ vez + complicado
- Salto semántico: dif. el operacione HLL y op de org.

Todo esto conduce a:
- Repertorio de instrux grandes + modd
- Varias sentencias de HLL implementados en el hardware

Características de ejecux: estudios sobre programs escritos en HLL
- op realizadas: funcionamiento del procesador e interax c/ memoria
- op usadas: tipos y frecuencia de uso(organizax de mem y modd).
- Secuencia de ejecux: org. y control del compilador.

- estudios dinámicos: medir durante la ejecux

Operaciones: asignax, condicionales (IF, loop), el procedimiento llamada/retorno consume tiempo, algunas instrux HLL ayudan a reducir a muchas op de máquina

Operandos: ppal variables escalares locales: la optimizax debe concentrarse en el acceso a las var locales

Scanned with CamScanner

github.com/ssofiaavila

llamadas y parámetros: se consume mucho tiempo, depende del nro. de pmo parametros tratados, depende del nivel de anidamiento.

La mayoría de los program. no tienen larga secuencia de llamados seguida x la correspondiente secuencia de retornos.

La mayoría de las var. son locales, la ref. a operandos están muy localizadas

Conclusiones: se puede ofrecer mejor soporte p/ HLL optimizando las prestaciones de las caract. + usadas y q' + tiempo consumen.
Usar gran nro de registros: optimizar fuente de operandos.
Poder cuidadoso atento al diseño de las cauce de instrux. (bifurcax, predic, etc).
Recomendable un repertorio simple de instrux

Ampl. func de registros

- Aproximax x software: el compilador es necesario p/ asignar registros.
  - Asignax de registros a las variables q' se usan + en un periodo de tiempo dado.
  - Requiere el uso de algoritmos de análisis de programa
- Aproximax x hardware: uso + registros, + variables pueden mantenerse en reg. durante periodos de tiempo + largos.

Registros p/ variables locales: muchos registros => reduce acceso a memoria.
Almacena var. escalares locales en registros.

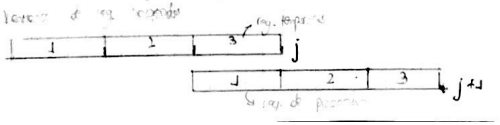
Problema: cada llamada de proc./función cambio la localidad.
Deben ser pasados los parametros, ptos. deben ser devueltos, las var. del programa deben ser devueltas.

Ventanas de registros: se requieren pocos parametros y var. locales en c/ llamado.
Hay limitar en la "profundidad" de llamados.
Usa múltiples conjuntos de pequeños reg. p/ c/ llamada <>
La llamada no cambia el conj. de reg. a usar.

Los retornos vuelven a cambiar al anterior conjunto de reg. utilizado

Hay 3 áreas dentro de un conj. de reg: 1- Reg. de parametros.
2- Reg. de D locales.
3- " temporales.

Los reg. temporales de un conjunto se solapan c/ reg. de parametros del niv. + bajo adyacente.
Posibilitar q' los param. sean pasados si q' existe transf. de D.



Tengo hasta 6 ventanas. si quiero añadir una última, se sobreescribirá en la v0 y lo de v0 se transferirá a memoria. Terminada la 7ma. subiendo, a v0 volveré lo q' fue transferido a memoria

Variables globales

El compilador asigna posiciones de memoria a las variables, ineficiente p/ var glob. o las q' se crean frecuente.
Incorporar al procesador un conjunto de reg. p/ las globales, divide reg de la ventana en cuatro.

Ampl. tam de reg vs. caché

Reg	Caché
Todas las D escalares locales	D escalares recientemente usadas.
Var. individuales	Bloques de memoria.
Var. glob asignada x el compilador	las glob y las usadas x el compilador.
Salvaguarda/restauran basadas en profundidad de anidamiento.	Salvaguarda basada en algoritmo de reemplazo.
Direccionamiento de registro	Direccionamiento de memoria

Optimizar uso de reg basados en el compilador.
Supongamos un pequeño nro del reg. El uso optimizado es responsabilidad del compilador.
Los program. HLL no tienen referencias explícitas a los reg.
Almacenado en C-

Cada entidad del programa candidata se asigna a un registro simbólico o virtual.
Asignar el nro. ilimitado de reg. simbólicos a un nro. fijo de reg reales.

Reg simbólicos q' no se solapan pueden compartir el reg real.

Si se agotan los reg. reales, algunos de los va se asignan en posic. de memoria.

Simplificar del compilador? Instrux complejas son difíciles de aprovecha. La optimizac es + difícil. Q tomará + velocidad.

Programas + pequeños? el program. ocupa - memoria, pero hoy en día la mem. es barata.

El nro de bits de memoria que ocupa no tiene porque ser + pequeño al tener - instrux.
+ instrux necesitan cód. de op + largo.

Programas + rápidos? programs q cada instx + sencillas, UC + complejo, mem control del microprogram + grande, + el tiempo de ejecux de instrux simple.

una instrux ciclo x reloj - op. reg o reg - modo sencillos - formato instrux sencillo diseño cableado - formato instrux fijo - > tiempo de compilox

Scanned with CamScanner

github.com/ssofiaavila

Práctica 6.

Control y DATA son direcciones de partes mapeadas en memoria y son variables globales

- Control: .word32 0x10000
- Data: .word32 0x10008

Cargo los datos en registros mediante lwu $s0, Data($zero) / lwu $s1, Control($zero)

Proceso:
- 1 en control → interpreto el valor de DATA como un nro 7L sin signo y lo imprime
- 2 control → Data = 7L c/signo y se imprimirá
- 3 control → Data pto. flotante e imprimirlo
- 4 control → Data tiene dirección de una cadena terminada en 0, imprimo string
- 5 control → DATA+5 coordenada X, se pinta el pto indicado. Resolución 50x50.
- 6 control → limpio pantalla alfanumérica
- 7 control → gráfico
- 8 control: permite ingresar. Nro 7L o pf: y el valor se podría leer en DATA.
- 9 control → se podría pintar un color y en data se leerá el código ASCII de dicha tecla en DATA. Debo leer DATA con lbu

Pintar pixel 1) Poner en DATA+3 el color RGBA (4 bytes)
2) DATA+4 coordenada Y
3) DATA+5 coordenada X

1) .data
CONTROL: .word32 0x10000
DATA: .word32 0x10008
texto: .asciiz "

.code
lwu $s0, DATA($zero)
lwu $s1, CONTROL($zero)
daddi $t0, $zero, texto
daddi $t2, $zero, 9
daddi $t4, $zero, 7A
loop:
sd $t2, 0($s1)
lbu $s2, 0($s0)
beq $s2, $t4, fin
sb $s2, 0($s0)
daddi $t0, $t0, 1
ld $t3, 0($t0)
daddi $t2, $t2, -1
bnez $t2, loop
fin: halt

2) .data
CONTROL: .word32 0x10000
DATA: .word32 0x10008
.code
lwu $s0, CONTROL($zero)
lwu $s1, DATA($zero)
jal ingreso
sltiu $t0, $v0, 0
beqz $t0, muestra
halt
ingreso: daddi $s2, $zero, 8
sd $s2, 0($s0)
ld $v0, 0($s1)
jr $ra
muestra: daddi $t0, $zero, 8
dmulu $t1, $v0, $t0
lwu $t0, pres($t1)
sd $t0, 0($s0)
daddi $t0, $zero, 5
sd $t0, 0($s1)
jr $ra

3) .data
CONTROL: .word32 0x10000
DATA: .word32 0x10008
PIDO: .word 0
.code
lwu $s0, CONTROL($zero)
lwu $s1, DATA($zero)
jal ingreso
daddi $a0, $zero, $v0
daddi $a1, $zero, $t1
jal escribir
halt
escribir: daddi $t0, $a0, $a1
sd $t0, 0($zero)(DATA)
daddi $t0, $zero, 1
sd $t0, 0($zero)(CONTROL)
jr $ra

.asciiz "cero"
.asciiz "uno"
.asciiz "dos"
.asciiz "tres"
.asciiz "cuatro"
.asciiz "cinco"
.asciiz "seis"
.asciiz "siete"
.asciiz "ocho"
.asciiz "nueve"
correcto: .word "BIENVENIDO"
incorrecto: .word "ERROR"

4) .data
CONTROL: .word32 0x10000
DATA: .word32 0x10008
clave: .word "AE10"
.code
lwu $s0, CONTROL($zero)
lwu $s1, DATA($zero)
jal chor
lbu $t3, clave($zero)
jal comparar
daddi $v0, $zero, 0
chor: daddi $t0, $zero, 4
loop: daddi $t1, $zero, 9
sd $t1, 0($s1)
ld $t2, DATA($t0)
daddi $v0, $v0, $t2
daddi $t0, $t0, -1
bnez $t0, loop
j $ra
daddi $t3, $zero, 0
daddi $t4, $zero, 0
comparar: daddi $t0, $zero, 4
suma: ld $t1, CLAVE($t3)
daddi $t3, $zero, 8
daddi $t4, $t4, $t1
daddi $t3, $t3, 8
daddi $t0, $t0, -1
bnez $t0, suma
beq $v0, $t4, correcto
lwu $t0, incorrecto($zero)
sd $t0, 0($s1)
daddi $t0, $zero, 4
sd $t0, 0($s0)
correcto: ld $t0, correcto($zero)
sd $t0, 0($s1)
daddi $t0, $zero, 4
sd $t0, 0($s0)

Scanned with CamScanner

github.com/ssofiaavila

BB
BB
SOO

Winmips procesador tipo RISC. Instrucciones de 32 bits, registros de 64 bits. No hay flags de estado. Arquitectura hardvard.

Tiene 32 reg de ZL de prop gral. y 32 de PF. f0 es un reg cualquiera

Throughput → incrementa productividad pero no reduce tiempo de ejecuc

Podemos hacer op de E/S mediante el mapeo.

E/S $ DATA y disp para valor en control

control y DATA son celdas de memoria / se leen y escriben con instruc usu lww

string c/ \n hace un salto de linea

RGB → 3 colores. 24 bits → 6 millones colores. $2^8 \rightarrow 255$ colores por pixel $2^8 \times 3 = 6$ mill.

↑ pantallo gráfica

Clase 7

Sistema de memoria: los programadores desean acceder a cant. ilimitada de memoria rápida.

La solución es la jerarquía de memoria. Organizado en niveles q son abordados en <> logros físicos diferentes, fabricados c/ tecno <> q se gestiona de manera independiente

Objetivo: la velocidad del sist. deberá ser, la del niv. más rápido al costo del niv. más barato.

A medida q nos alejamos del CPU, cada niv. inferior es + grande, + lento y + barato que el niv previo o superior en la jerarquía.

Debe haber correspondencia de direcciones en los <> n.v.
Propiedades a cumplir

Inclusión: los D almacenados en un niv han de estar almacenados en los niv. inferiores a él

Coherencia: las copias de la misma info. en los <> niveles deben contener los mismos valores.

¿Por qué funciona la jerarquía? Si obj es conseguir una estructura de gran capacidad c/ la velocidad del nivel + bajo de jerarquía para c/ uno

Principio de localidad de referencias: tolerancia al niv + rápido

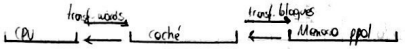
- Localidad temporal: los elementos de memoria referenciados recientemente, volverán a usarse en un futuro próximo. ⇒ subo acord de niv.
En vez de traer una palabra de MP, trae un bloque
- Espacial: los elem. de mem. cuyas direc están próximas a los últimos referenciados serán referenciados ⇒ subo un bloque de niv

fabricado c/tecno magnética y controlado x SO. c/privilegio

caché – MP – MV

c/ formado x memoria estática controlados x controladores del caché hay varios tipos

Memoria caché: cantidad de pequeño de memoria se ubica e/ la memoria ppal y CPU. Puede localizarse en un chip o en módulo CPU



Funcionamiento de la caché: la CPU solicita contenido de 1 direc de memoria.

La caché tiene ese dato? · Si es así, lo obtiene de la caché (rápido)
· Si no está, se lee el bloque q contiene esa direc desde la memoria ppal y copia en la caché (sp), la caché entrega el D requerido a la CPU

La caché incluye etiquetas p/ identificar qué bloque de la memoria ppal está en c/ línea

Conceptos básicos: · Acierto (hit): se encuentra en la caché el D solicitado.
· Fallo (miss): no se encuentra en la caché el D solicitado

Un bloque q contiene la palabra accedido se copia de la memoria ppal a una línea de caché.

· Tiempo p/ servir un fallo: depende de la latencia y ancho de banda de la mem ppal. El tiempo e/ q determino q el bloque no está en el caché y hallo el correcto.
Latencia: tiempo p/ completar un acceso a memoria
Ancho de banda: cant de info x unidad de tiempo q puede transferirse desde/hacia mem.

· Fallos de caché: se gestionan mediante hardware y causan q el procesador se detenga hasta q el D esté disponible

Prestaciones de jerarquía: $T_{acceso} = T_{aciertos} + T_{fallos_memoria}$

$T_{fallos_memoria} = Tasa\ de\ fallo \times penalización\ de\ fallo$. ⇒ $T_{acceso} = T_{acierto} + TF + PF$

P/ mejorar las prestaciones · Reducir el tiempo de acceso en caso de acierto
· ↓ tasa de fallos
· ↓ penalización de falla

Supongan proc. de reloj 2 GHz y CPI = 1. ejecuta cód de 500 instruc.

Caso 1: hay 2 mem. caché ideales M1 y MD (no habrá fallo y t_{acceso} despreciable)

$t_1 = t_{CPU} + t_{mem} = t_1 = t_{CPU} + 0 = t_{CPU}$

$t_1 = nI \cdot CPI \cdot T = 500 \cdot 1 \cdot (1/2G) = t = 250\ ns.$

3 tipos de fallo de caché: miscelánea → los 1° ref. referenciado una cantidad estimulares de capacidad → cuando no caben todos los bloques en caché deben producirse reemplazos de conflicto → todos bloques c/= localización en caché y se van reemplazando.

Scanned with CamScanner

github.com/ssofiaavila

tamaño de bloque, bloque grande, paralelismo → falla. La solución es 2 niv de caché
L1 → + cercano al procesador, pequeño y rápido
L2 → > tamaño y + lento

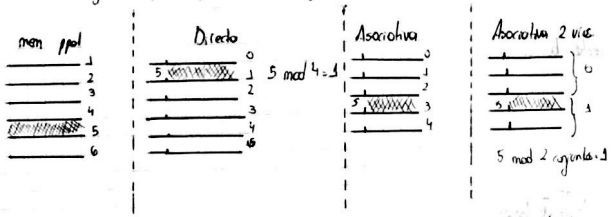
Diseño de la caché: organización (tamaño y capac), política de ubicación (tipo f(x) de correspondencia), política de reemplazo (algoritmo de sustitución), política de escritura

- Organización → tamaño caché y niveles. → tamaño de caché ↗ → pequeño: ofrece tasa de fallos para programas grandes pq ↗ el espacio integrado, caracteres cortos.
- Ubicación de un bloque: y caché ↑ grande = ↑ lento

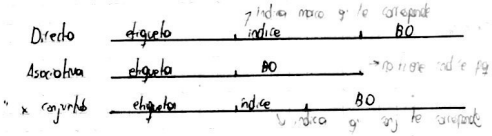
- Correspondencia directa: un bloque ↑ solo puede estar almacenado en un lugar de caché.
N° línea caché = n° bloque (ref) mod n° líneas caché.

- "total" asociativo: un bloque puede almacenarse en cualquier lugar de la caché

- asociativa x conjuntos: un bloque puede almacenarse en un conj. restringido de lugares en la caché. Un conjunto es un grupo de líneas de la caché.
N° conj = nº bloque (ref) mod n° conj. caché



La interpretación de la dir física depende del tipo que se use. Indica indirecta la línea o el conjunto q le corresponde. BO representa todos los dirs q pertenecen al bloque



Correspondencia directa: simple, poco costoso. Hay una posición correcta p/ c/ bloque dado: si un programa accede a 2 bloques q se corresponden a la misma línea. (<> bloques de mem. ppal.) de forma repetida: la pérdida de caché serán grandes

- asociativo: un bloque de MP puede colocarse en cualquier línea de caché. La etiqueta identifica unívocamente un bloque de mem.
Todas las etiquetas de la línea se examinan p/ buscar una coincidencia.
Búsqueda costosa

- asociativa de 2 vías: combina lo mejor de otras correspondencias. La caché se divide en un conj. de grupos c/ conj. tiene un nro. de líneas.
Un bloque determinado corresponderá a cualquier línea de un conj determinado

↑ p/ decidir q' ocurre c/ un fallo
Política de reemplazo: algoritmos de sustitución en correspondencia directa - no asociativa (LRU - FIFO - LFU - aleatorio)

Algoritmo de sustitución: en correspondencia directa No hay elección. Sólo hay una posible línea p/ c/ bloque. Se necesita una sustitución de esa línea (si ocupada).

- Correspondencias asociativas: los algoritmos deben implementarse en hardware (p/ > velocidad).

- LRU (menos recientemente usado): requiere controles de tiempo. Uso campo p/ saber cuántas veces usé una línea

- FIFO (primero en entrar - primero en salir): requiere control de acceso. Se sustituye aquella línea q' ha estado + tiempo en la caché. Requiere etiqueta de E/S

- LFU (menos frecuentemente usado): requiere control de uso. Se sustituye aquella línea q' ha experimentado - referencias. Requiere control de tiempo

- Aleatorio: se sustituye línea al azar.

Política de escritura: se debe evitar inconsistencias en el caso de escritura.
Tener en cuenta:
La CPU escribe sobre una línea de caché (el bloque de MP correspondiente debe ser actualizado en algún momento). Un módulo de E/S puede tener acceso directo a la MP.
En procesamiento paralelo, las múltiples CPU pueden tener cachés individuales

Política de escritura: en acierto:
Write-through (escritura inmediata): se actualizan simultáneamente la posición de la caché y de la MP. Con múltiples CPU, observar el tráfico a memoria ppal p/ mantener actualizada c/ caché local. Se genera mucho tráfico y retraso escritura

Write-back (post-escritura): la info. sólo se actualiza en caché. Se marca como actualizada => bit de "sucio". La MP se actualiza en el reemplazo y puede contener info. errónea en algún momento. Puede hacer escrituras simultáneas

Política de escritura, en fallo:

Write allocate: la info. se lleva de la MP a caché. Se sobrescribe en la caché. Habitual c/ write-back

No-write allocate: el bloque no se lleva a la memoria caché. Se escribe directo a MP. Habitual c/ write-through

Scanned with CamScanner

github.com/ssofiaavila

Pentium 4: - hasta 3 niv. de caché. Caché L1 separado de D e instrux.

- Caché de D (8 Kbyte): acceso a conj de 4 vías. Bloques de 64 bytes. Escritura: write-back. Acceso D obtenido en 2 ciclos reloj

La caché de instrux almacena segmentos de camino de ejecux de instrux decodificadas

- Caché L2: unificada p/ D e I. 256 Kbyte. Org asociativa a 8 vías. Bloques 128 bytes. Política post escritura. Latencia acceso de 7 CPR.

Las 2 cachés en chip de procesador. Ancho de banda p/ L1 y L2 de 48 Gbytes/s.

La org. permite L3 en el mismo chip

Stalling cap. 6. — Bellaví cap 2 apartado 2.1 a 2.4.

Clase 6

Procesador escalar: ejecux secuencial de instrucciones: | F | D,RW | E,P/ALU | E,D/ALU | (F,D,ALU: I1; E,P/ALU: I2; E,D/ALU: I3)

Segmentado de cauces: unidad segmentada es una secuencia de etapas c/ la propiedad de que nuevas operaciones puedan iniciarse mientras otras están en proceso

Procesador escalar segmentado: gráfica

Mayores prestaciones: la evolución lógica de los diseños de cauces segmentados d.o lugar a la aparición de 2 términos de ejecux de + prestaciones:

1) Procesadores supersegmentados 2) Procesadores superescalares.

Enfoque superSegmentado: muchas op no ejecutan en un ciclo de reloj. Obtendremos > prestaciones subdividiendo el ciclo de reloj en sub-intervalos. Resulta una mayor frecuencia del reloj de reloj. División de las etapas mayor del cauce segmentado en sub-etapas + pequeñas (y internas + rápidas) y se transmiten D a la > velocidad del reloj de reloj

El tiempo p/ las instrucciones individuales no varía. ↑ el grado de paralelismo, ↑ la aceleración percibida

Enfoque superescalar: se pueden llevar a cabo + de 1 instrux simultáneas. Conlleva la duplicación de algunos o todos los puertos del CPU/ALU. p/ captar múltiples instrucciones al mismo tiempo, decodificarlas simultáneas, ejecutar código y almacenarlo, mientras se lleva a cabo una ejecux ALU.

El grado de paralelismo y aceleración aumentan ya q' se ejecutan + instrux en paralelo

Paralelismo de instrucciones: "existe cuando instrucciones de una secuencia son independientes" y pueden ejecutarse en paralelo solapándose.

Limitaciones: dependencia de D, dependencia relativa al procedimiento, conflicto de RR, dependencia de S, antidependencia

Paralelismo de la máquina: medida de la capacidad del procesador p/ sacar partido del paralelismo de instrucciones. Depende del nro de instrux replicadas x ciclo, nro unidades funcionales, mecanismos p/ localizar instrucciones independientes.

Identificar paralelismo y organizar F,D,E en paralelo. Renombre de registros, ventana de instrux

Sobre las instrucciones en la localizax de instrucciones independientes: orden en que se captan las instrucciones, orden q' se ejecutan, orden q' se actualizan los reg y posiciones de memoria

Uso óptimo del cauce: atender independencias.

Política de emisión en procesadores superescalares: emisión y finalizax en orden, emisión en orden y finalizax desordenada, emisión y finalizax desordenada

Riesgos: 1) RAW (dependencia verdadero)
la salida de (1), R_3, es un operando en (2)

```
(1) R3 := R3 op R5
(2) R4 := R3 + 1
(3) R3 := R5 + 1
(4) R7 := R3 op R4
```

2) WAW (dependencia de salida)
(3) y (3) intentan escribir en R_3

2) WAR (antidependencia): instrux (3) no puede comenzar hasta q' la instrux (2) haya obtenido uno de sus operandos (R_3)

Renombre de registros: las dependencias de salida y antidependencias surgen pq' los valores de los registros no pueden reflejar la secuencia de valores dictada x el flujo del programa

Esto puede detener alguna etapa del cauce. Cuando la ejecux de una instrux quedó en stand-by, se almacena en un reg nuevo => renombramiento de reg.

Los reg se asignan automáticamente, o sea, las referencias posteriores son los registros nuevos

Implementación superescalar: estrategia de captar simultáneas de múltiples instrux.
Lógica p/ determinar dependencias verdaderas e/ valores de registros y mecanismos p/ comunicar esos valores

Mecanismos p/ iniciar o emitir múltiples instrucciones en paralelo. RR p/ la ejecux en paralelo de múltiple instrux.
Mecanismos p/ entregar el E del procesador en un orden correcto

Consideraciones deseable en el proc superescalar:

- Influencia de las excepciones. En el momento en q' se produce una excepx hay varias instrux en ejecux.
  Si I_1 produce una excepx

Excepciones imprecisas => el comportamiento obtenido sea idéntico al que tendría lo mismo

Scanned with CamScanner

github.com/ssofiaavila

computadora no segmentada.

P/ garantizar un estado consistente (preciso): instrux anteriores terminan correctam', la q' origina la excepc y siguiente se abortan, tras lo rutina de tratamiento se comienza x la q' originó la excepc.

Excepciones precisas: ej: los ptos. Se almacenan en el ptdor en q' apuntaran las instrux.

Retarda el W2 hasta ciclo 6. Con las mismas interrupciones externas => excepciones precisas.

La unidad de emisión deja de emitir y se vacía la cola.
Todas las instrucciones pendientes se completan => comienza el proc de interrupc

Compromiso entre: • Ejecux desordenada ↔ libera las unidades de ejecuc
• Completar instrucciones en orden ↔ excepciones precisas

Posible solux emisión desordenada y finalizac ordenada → hace problemas
↓ elimina excepciones
etapa de escritura temporal (TW) → reconocimiento de reg.

El reg temporal TW: se usa solo x inst posteriores a i

Cap 13 Stalling; cap 3 Hennessy

Clase 9:

Buses: buses del sistema: memoria, módulo de E/S, procesador.

- La memoria lee/escribe D, recibe direcciones y señales de control como reloj o L/E.
- Módulo E/S: recibe y entrega D, interactúa c/el procesador y memoria. Recibe direcc y señales de control (recibe y genera ej: DMA)
- Procesador: L/E en memoria, genera/recibe señales.
- Bus de D: ancho de bus s/cant. se determina cantidad transferida n D simultáneo
- Bus de direcc: p/ubicar D o instrux. Su ancho determina capacidad de transferencia
- Bus de control: señales de UC y otros componentes
- Un bus multiplexado sirve p/ bus de D. e Instrux. Requiere un hardware de control y como resultado de usar un bus p/ D e I, el mismo será más chico debido a - circuitos y un sist. + chico.
- Arbitraje: coordina de acuerdo: DIS controla acuerdo → controlada DMA.

El control del bus de D puede ser centralizado o distribuido
Centralizado: una única unidad controla
Distribuido: cualquiera puede nombrarse dueño y coordina

vertical / horizontal => estructura de D

Clase 9- procesamiento paralelo → Arquitectura c/ varios procesadores
Sea cual sea el nivel de prestaciones, la demanda de máquinas de > rendimiento seguirá existiendo
- Mejorar el rendimiento de una máquina c/ un solo procesador → paralelismo a niv. instrux.
- Arquitecturas de sist. c/ varios procesadores → paralelismo a niv. proceso

Categorías de computadoras: • SISD: una secuencia de instrux y 1 secuenc. de D
• SIMD: una secuenc. de I y múltiple secuenc. de D.
• MISD: múltiples secuencias de I y secuencia de D
• MIMD: múltiples secuencias de I y múltiple de D

SISD: un único procesador interpreta una única secuencia de I. Los D se almacenan en una única memoria. Son monoprocesador.

SIMD: una única instrucción de máquina controla paso a paso la ejecux simultánea de un cjto nro de elem de proceso.
C/ elem tiene una memoria dedicada, c/I. es ejecutada x c/ procesador, c/un conjunto de D <>. Procesadores vectoriales y matriciales.

matricial? vectorial? paralelo?
Computadora c/ una única UC y una matriz de elem. "computacionales".
Tipos de instrucciones del procesador:
- Extensiones de la instrux escalares: sumar almacenar, multiplicar, etc. se convierten en op. vectoriales ejecutadas en todos los procesadores simultáneo'

Debe añadirse la capacidad de transferir al conj. de instrux los D escalares y vectoriales c/ procesador.

MISD: se transmite una secuencia de D a un conj. de procesadores. C/ procesador ejecuta una secuencia de instrux <>. Esta estruct. nunca se implementó.

MIMD: conj. de procesadores ejecuta secuencias de I <> en simultáneo. Conj. de D <>.
Se puede dividir s/ forma de comunicarse:

- Mem. compartida: SMP (multiprocesadores simétricos) y sist. NUMA
- Mem. distribuida: clusters.

Multiprocesador simétrico - SMP computadora autónoma c/

- 2 o + procesadores similares de capac. comparables
- Comparten la MP y los E/S
- Interconectados mediante un bus u otro tipo del sist. de interconexión.
- Tiempo de acceso a memoria similar p/ todos los procesadores
- Todos los proces. pueden desempeñar = fx.
- SO integrado, q' proporciona interfaz c/ proc. y programas

Scanned with CamScanner

github.com/ssofiaavila

Ventajas:
- Mejores prestaciones: si el trbjo. a realizar puede organizarse en paralelo.
- Buena disponibilidad: un fallo en un proc. no detendrá la comp.
- Crecimiento incremental: se pueden añadir + procesadores.
- Escalado: en fx a, la cant. de procesadores.
- Cuidado: bus compartido

Desventajas: bus tiempo compartido: prestax limitada x el tiempo de ciclo del bus.

Cada procesador debería estar equipado c/ una memoria caché p/ mejorar las prestaciones: sobrecarga el nro. de accesos.
Se pueden producir problemas de coherencia de caché. Este problema debe ser resuelto x el hardware → protocolo de sondeo y de directorio

Clusters: computadores completos interconectados q trabajan conjunto como un único AR.
c/ computador se denomina nodo. Prestaciones y disponibilidad elevada. Aplicax propio de un servidor. Alternativa a SMP

Beneficios: escalabilidad absoluta, incremental. Alta disponibilidad. Mejor relax precio/prestax

Clusters vs SMP
- Ambos: dan soporte a aplicaciones de + demanda de AR. Disponible comercial!
- SMP: + fácil de administrar y configurar. Ocupa a sist. de un solo procesador, le + ventajas es la dif. ppal., menor espacio físico, => consumo de potencia
- Cluster: superior escalabilidad incremental y absoluta. Superior disponibilidad, redundancia

Termino UMA, NUMA, CC-NUMA
todos los proc. tienen acceso a toda la mem.

- UMA - Uniform memory access: igual tiempo de acceso a todas las regiones de mem, = tiempo de acceso a memoria p/ <> procesadores
- NUMA - Nonuniform memory access: el t. de acceso difiere s/ reg de mem accedido, <> procesadores acceden a reg de mem en <> velocidades.
- CC-NUMA = caché coherente NUMA: es un NUMA q mantiene coherencia de caché c/ las caché de <> procesadores.

Motivos NUMA.

SMP tiene límite práctico de nro. de procesadores.
En clusters c/ nodo tiene MP. Coherencia de caché mantenida x software.
NUMA retiene aspect de SMP y brinda prox. o gran escala.

Obj NUMA: tener memoria transparente del sist. y permitir nodos, ej: c/ bus o sist. conex interno

Ejemplos CC-NUMA: c/ proc. tiene caché L1 - L2. c/ nodo tiene MP y están conectados los nodos c/ un tipo de red.
c/ proc. ve un único espacio de direcciones de memoria.
Orden de acceso a memoria → caché L1, L2, MP, MV.
Automático y transparente

Procesamiento multihebra: paralelismo de instrux: sin el aumento de complejidad y consumo de potencia de los segmentos de cause y superescalares.
La secuencia de instrux se divide en secuencias + pequeñas llamadas hebras q pueden ejecutarse en paralelo.
Amplia variedad de diseños multihebra

Términos: hebra y proceso:

Conjunto de hebras en procesadores multihebra puede no ser el de SO multiprogramado.

Proceso: un programa corriendo en una comp.

Prop. de RR: espacio de direcc virtuales p/ almacenar imagen de un proceso.
Planificax/ejecux: hoy camino de ejecux.

Conmutax de proceso

- Hebra: unidad de trbjo de un proceso q puede asignarse, incluso un contexto de proceso y cómo de D p/ su pila, se ejecuta secuencial.
Interrumpible: el procesador conmuta a otra hebra.

Conmutax de hebras: cambio de control del procesador c/ hebras de = proceso. Usual: menos costoso q la conmutax de proceso

Multihebra implícita y explícita.
Multihebra explícita: ejecux concurrente de instrux de <> hebras explícitas. Mezcla instrux de <> hebras en cauces computadores o x ejecux paralela en cauces paralelos.

Multihebra implícita: ejecux concurrente de varias hebras extraídas de un único programa secuencial, definidas estática x el compilador o dinámica x el hardware

Procesador multihebra: PC <> p/ c/ hebra q puedan ejecutarse concurrente
Se trata c/ hebra separada, pierde de saltos y renombre de reg
Aproximadamente c/ ejecux simultánea real

Cap 16 Stalling - Cap. 5 Guzmán

Scanned with CamScanner